

UNIT 3

Object-Oriented Design using UML



UML

- UML stands for “**Unified Modeling Language**”
- It is a industry-standard **graphical language** for **specifying, visualizing, constructing, and documenting** the artifacts of software systems
- The UML uses mostly **graphical notations to express the OO analysis** and design of software projects.
- **Simplifies the complex process** of software design

Why are UML Models Required?

- Modelling is an abstraction mechanism:
 - Capture only important aspects and ignores the rest.
 - Different models obtained when different aspects are ignored.
 - An effective mechanism to handle complexity.
- UML is a graphical modelling technique
- Easy to understand and construct

UML Diagrams

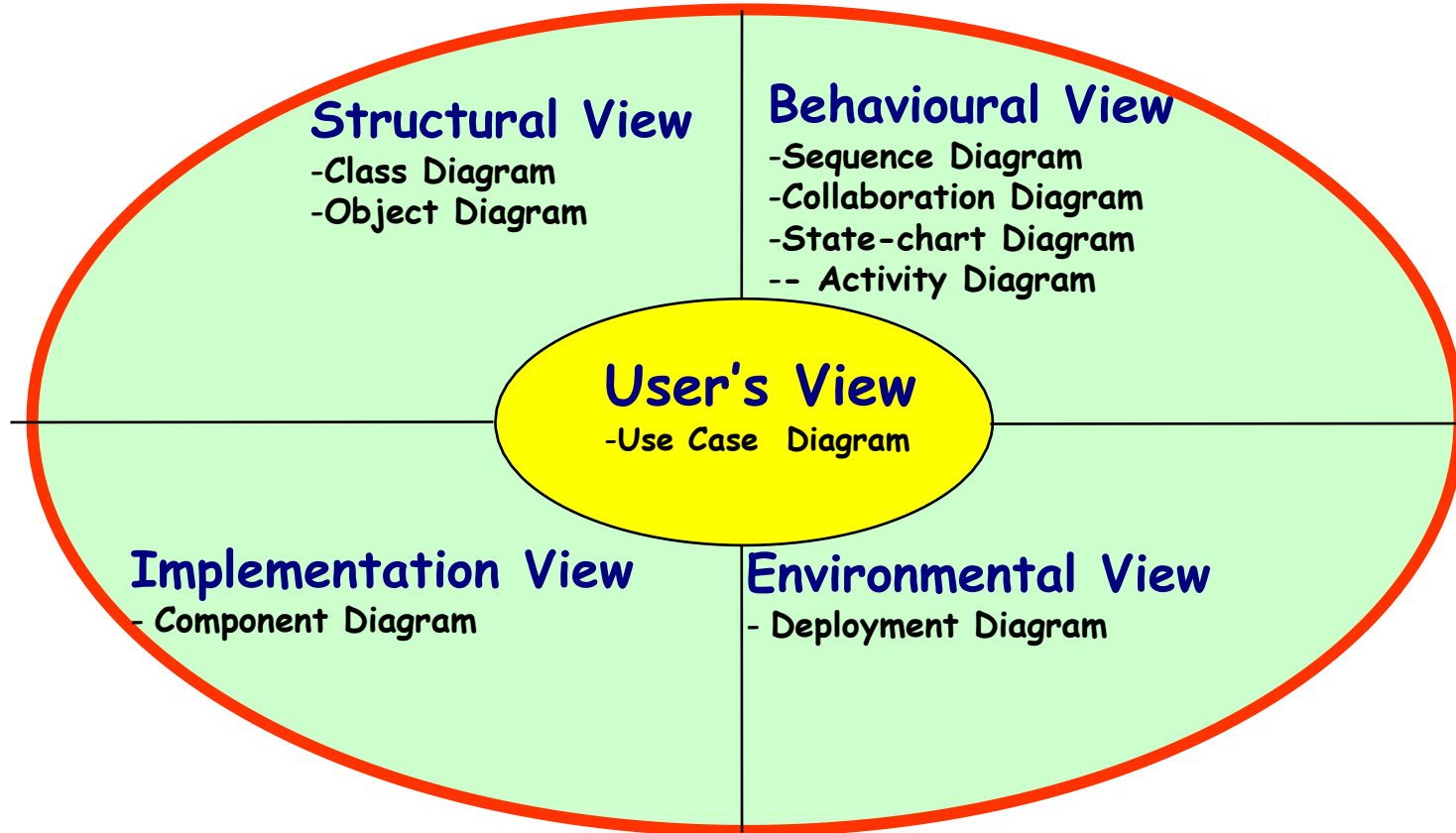
- Nine diagrams in UML1.x :
 - Used to capture 5 different views of a system.
- Views:
 - Provide different perspectives of a software system.
- Diagrams can be refined to get the actual implementation of a system.

● Views of a system:

- User's view
- Structural view
- Behavioral view
- Implementation view
- Environmental view

**UML
Model
Views**

9 Diagrams and 5 views in UML



Structural Diagrams

- **Class Diagram**
 - set of classes and their relationships.
- **Object Diagram**
 - set of objects (class instances) and their relationships
- **Component Diagram**
 - logical groupings of elements and their relationships
- **Deployment Diagram**
 - set of computational resources (nodes) that host each component.

Behavioral Diagrams

- **Use Case Diagram**
 - high-level behaviors of the system, user goals, external entities: actors
- **Sequence Diagram**
 - focus on time ordering of messages
- **Collaboration Diagram**
 - focus on structural organization of objects and messages
- **State Chart Diagram**
 - event driven state changes of system
- **Activity Diagram**
 - flow of control between activities

Are All Views Required for Developing A Typical System?

● **NO**

● For a simple system:

- Use case diagram, class diagram and one of the interaction diagrams only.

● State chart diagram:

- when class has significant states.

● Deployment diagram:

- In case several hardware components used to develop the system.

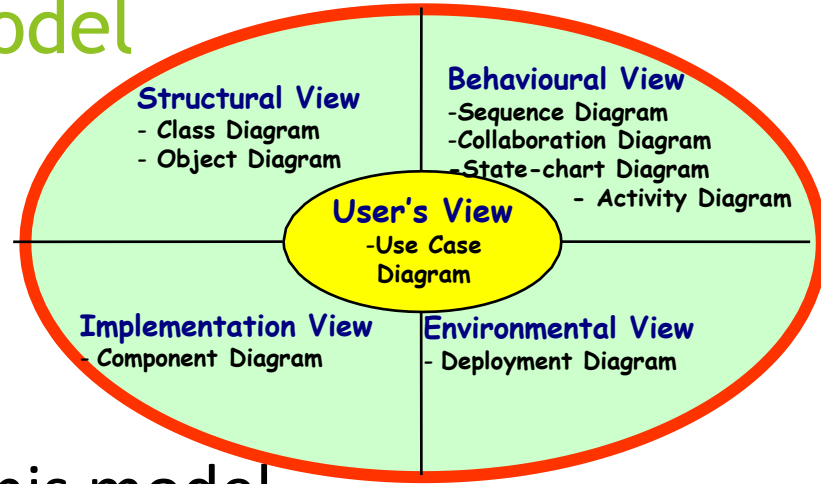
Some Insights on Using UML

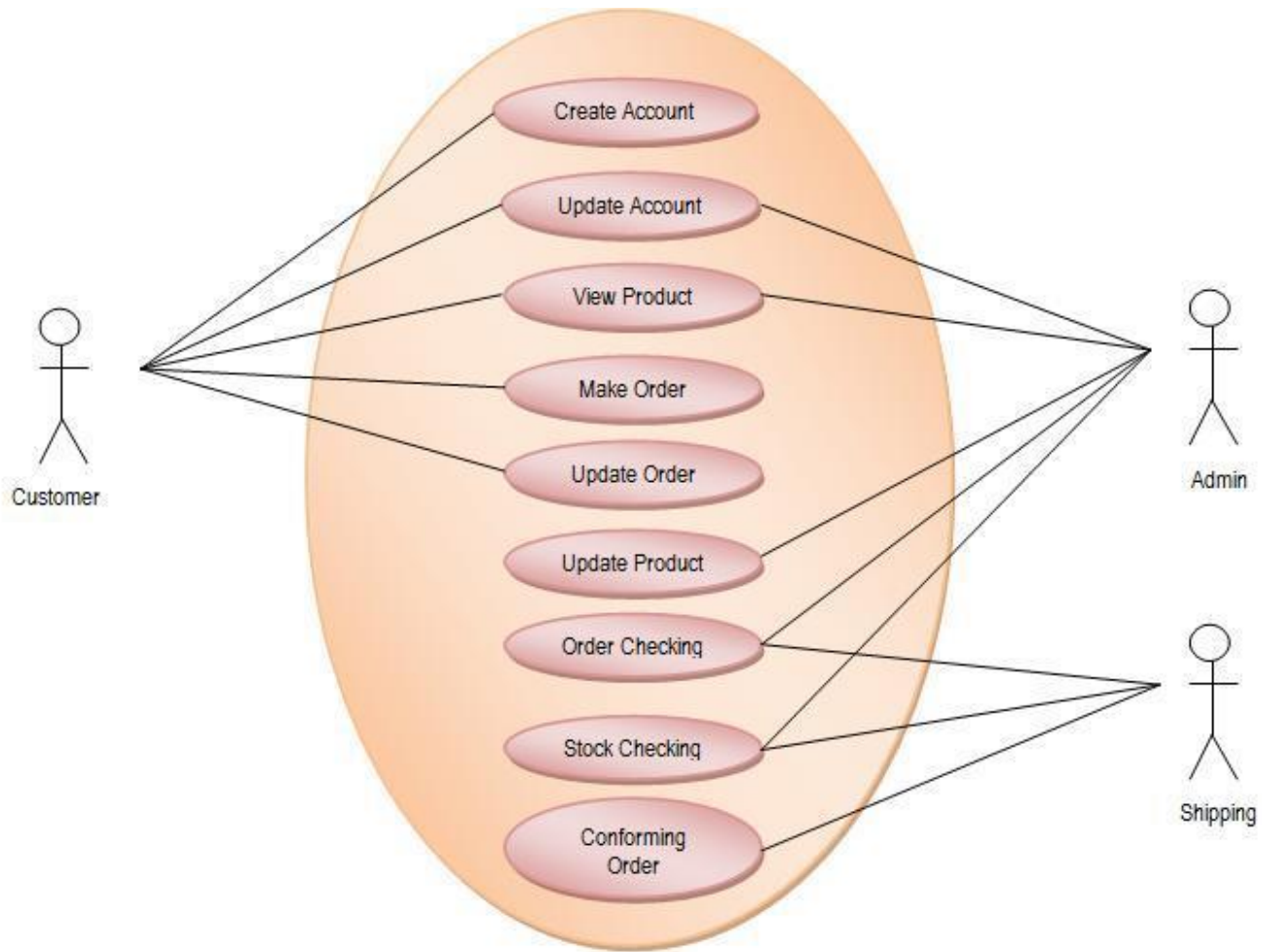
- “UML is a large and growing beast, but you don’t need all of it in every problem you solve...”
– Martin Fowler
- “...when learning the UML, you need to be aware that certain constructs and notations are only helpful in detailed design while others are useful in requirements analysis ...” Brian Henderson-Sellers

Use Case Modelling

Use Case Model

- Consists of a set of “**use cases**”
- It is the central model:
 - Other models must conform to this model
 - Not really an object-oriented model, it is a functional model of a system





A Use Case

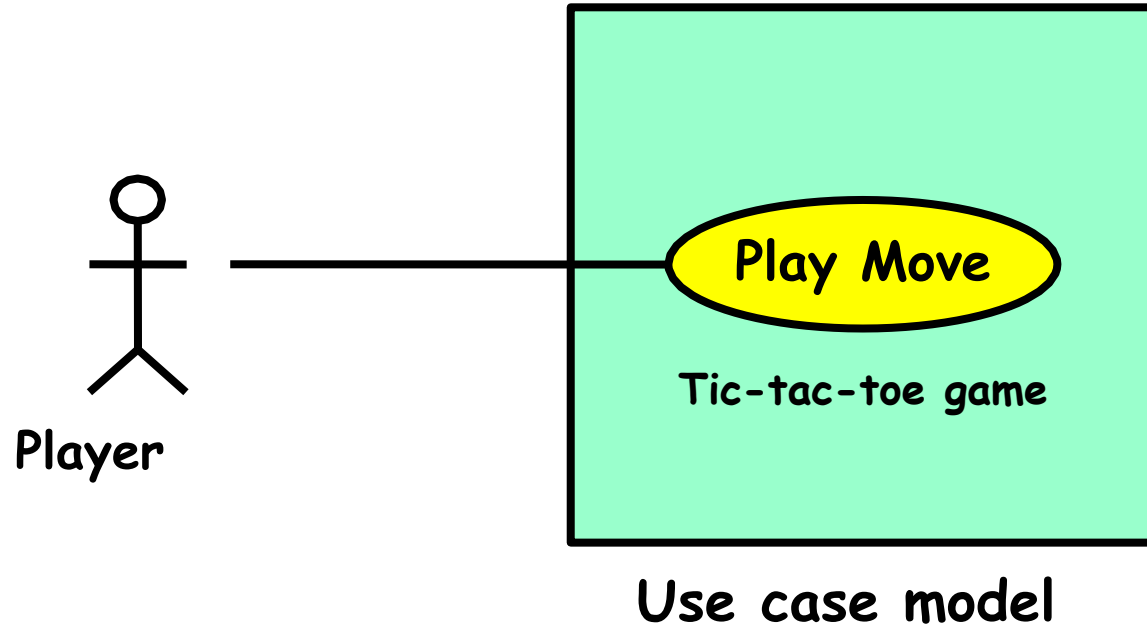
- **A case of use:** A way in which a system can be used by the users to achieve specific goals
- Corresponds to a high-level requirement.
- Defines external behavior without revealing internal structure of system
- **Set of related scenarios tied together by a common goal.**

—Use cases for a Library information system

- issue-book
- query-book
- return-book
- create-member
- add-book, etc.

**Example
Use Cases**

An Example Use Case Diagram

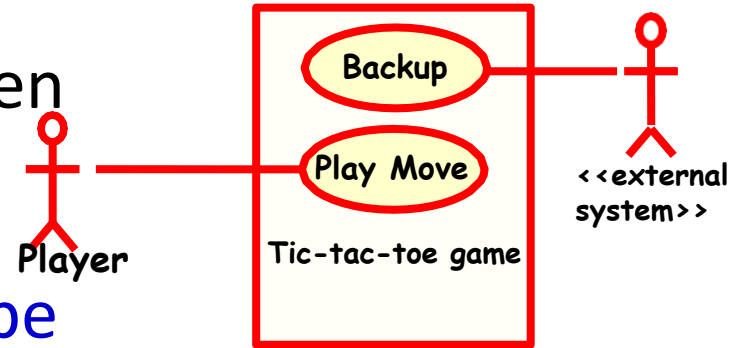


Why Develop A Use Case Diagram?

- Serves as requirements specification
 - How are actor identification useful in software development?
- Identifies different categories of users:
- Helps in implementing appropriate interfaces for each category of users.
 - Helps in preparing appropriate documents (e.g. **users' manual**).

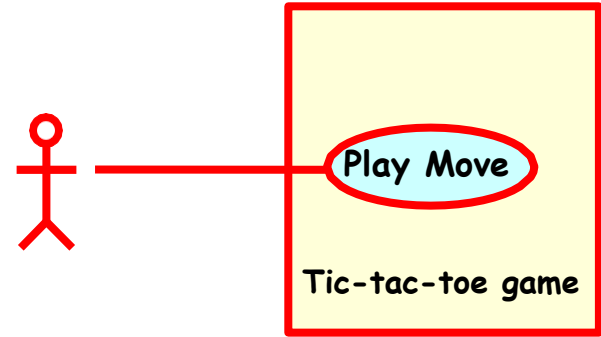
Representation of Use Cases

- Represented in a use case diagram
- A use case is represented by an ellipse
- System boundary is represented by a rectangle
- Users are represented by stick person icons (actor)
- Communication relationship between actor and use case by a line
- External system by adding a stereotype



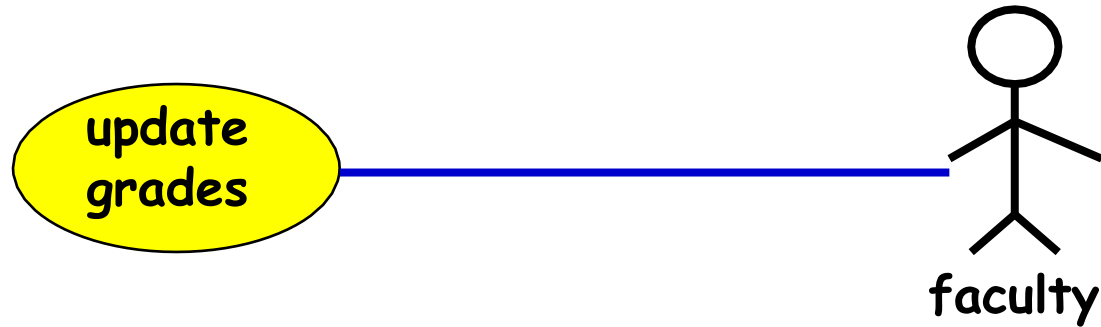
What is a Connection?

- A connection is an association between an actor and a use case.
- Depicts a usage relationship
- Connection does not indicate data flow

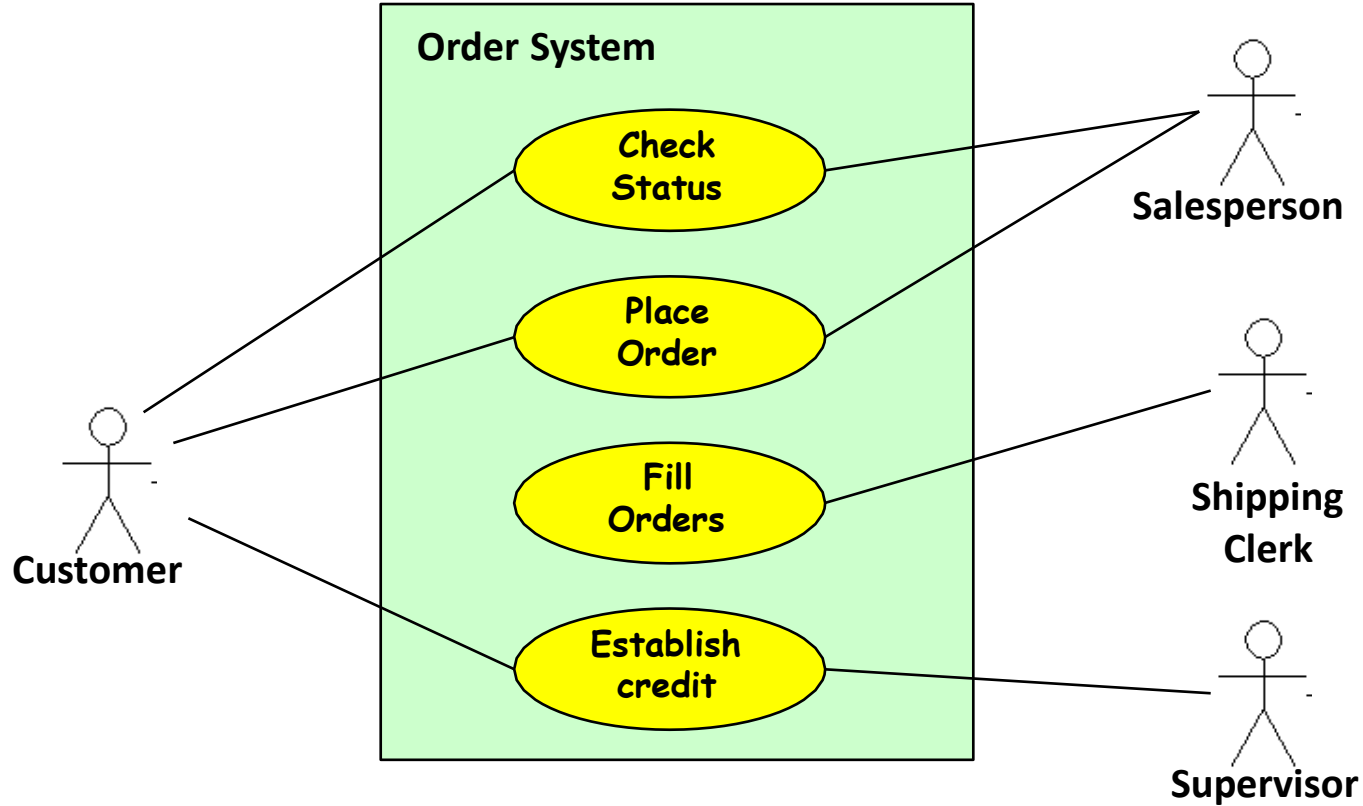


Relationships between Use Cases and Actors

- Association relation indicates that the actor and the corresponding use case communicate with one another.



Yet Another Use Case Example



Factoring Use Cases

- Two main reasons for factoring:

- Complex use cases need to be factored into simpler use cases
- Helps represent common behavior across different use cases

- Three ways of factoring:

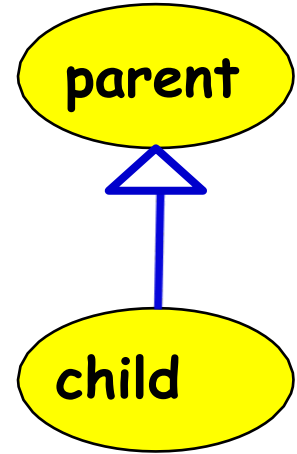
–Generalization

–Include

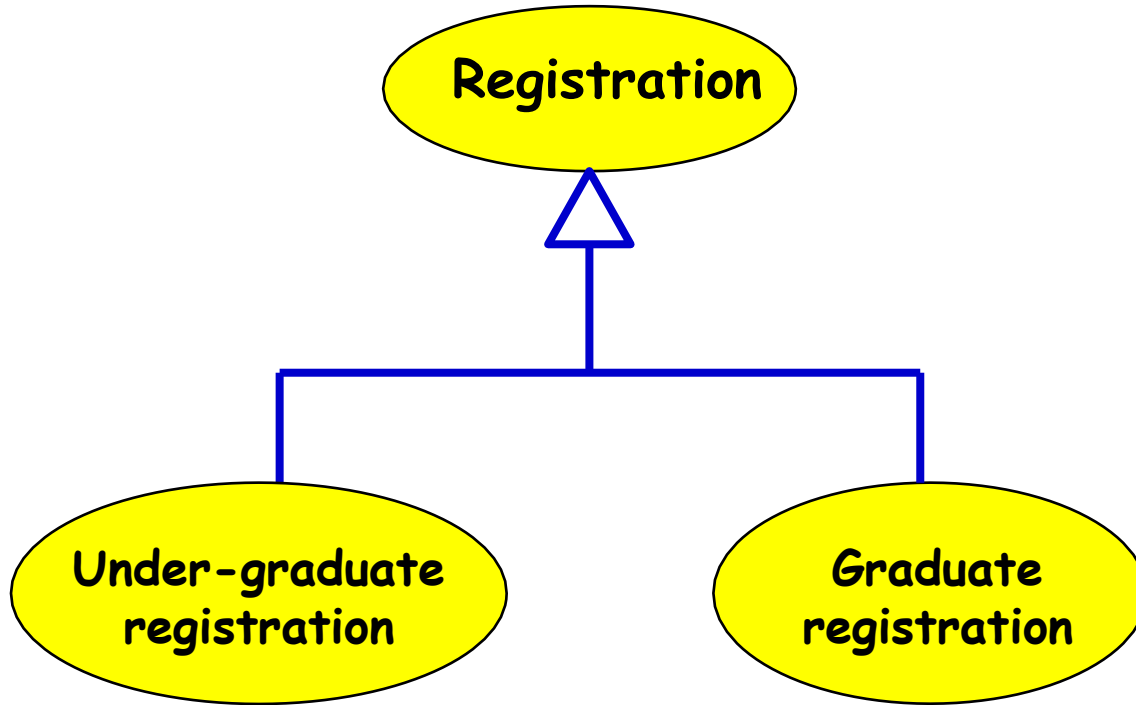
–Extend

Generalization

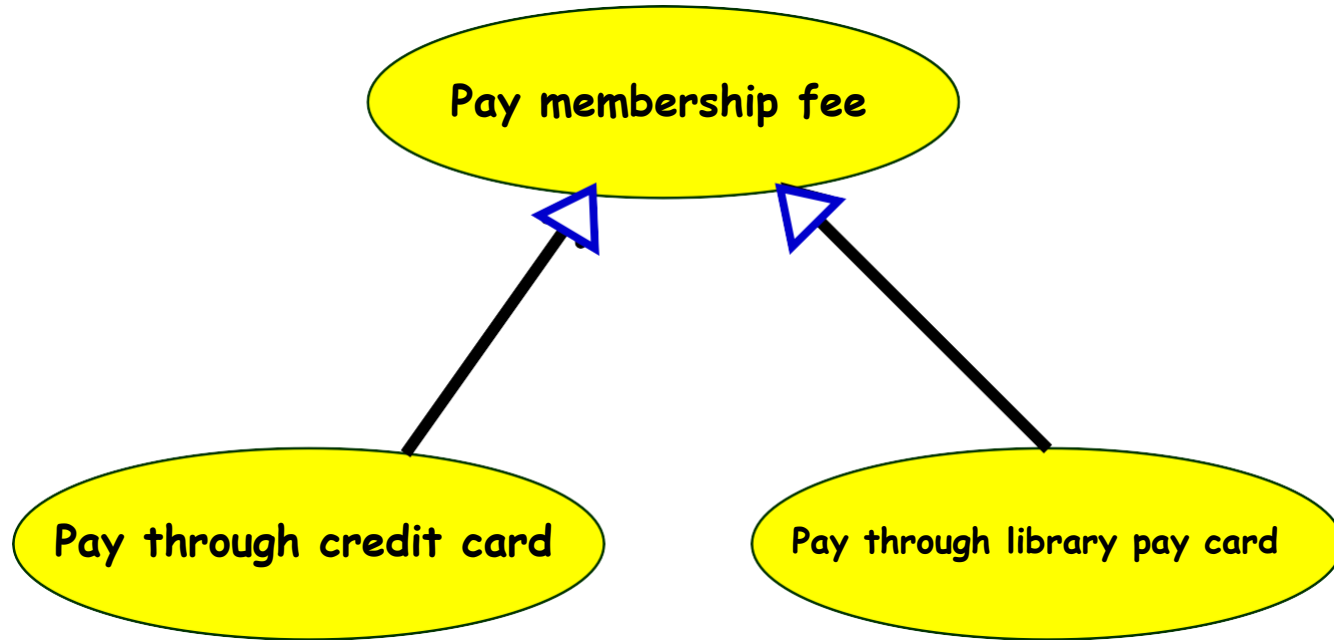
- The child use case inherits the behavior of the parent use case.
 - The child may add to or override some of the behavior of its parent.

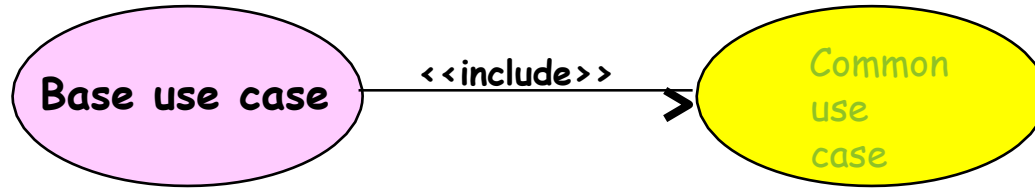


Generalization Example 1

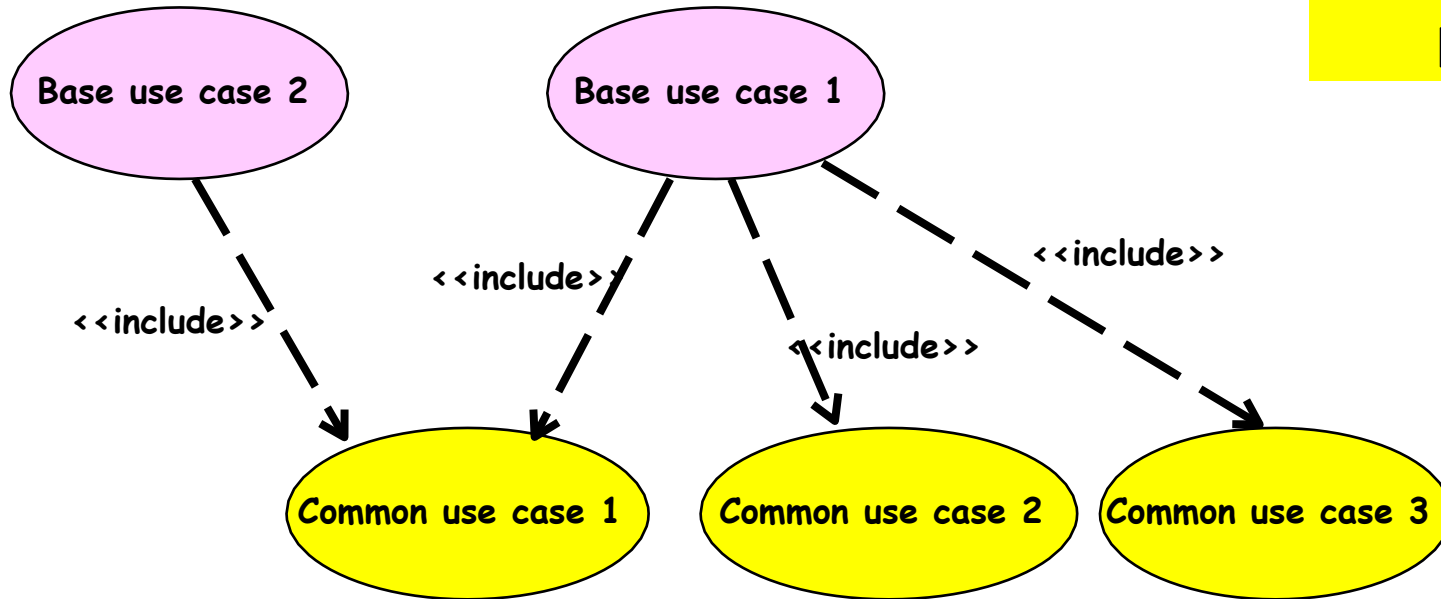


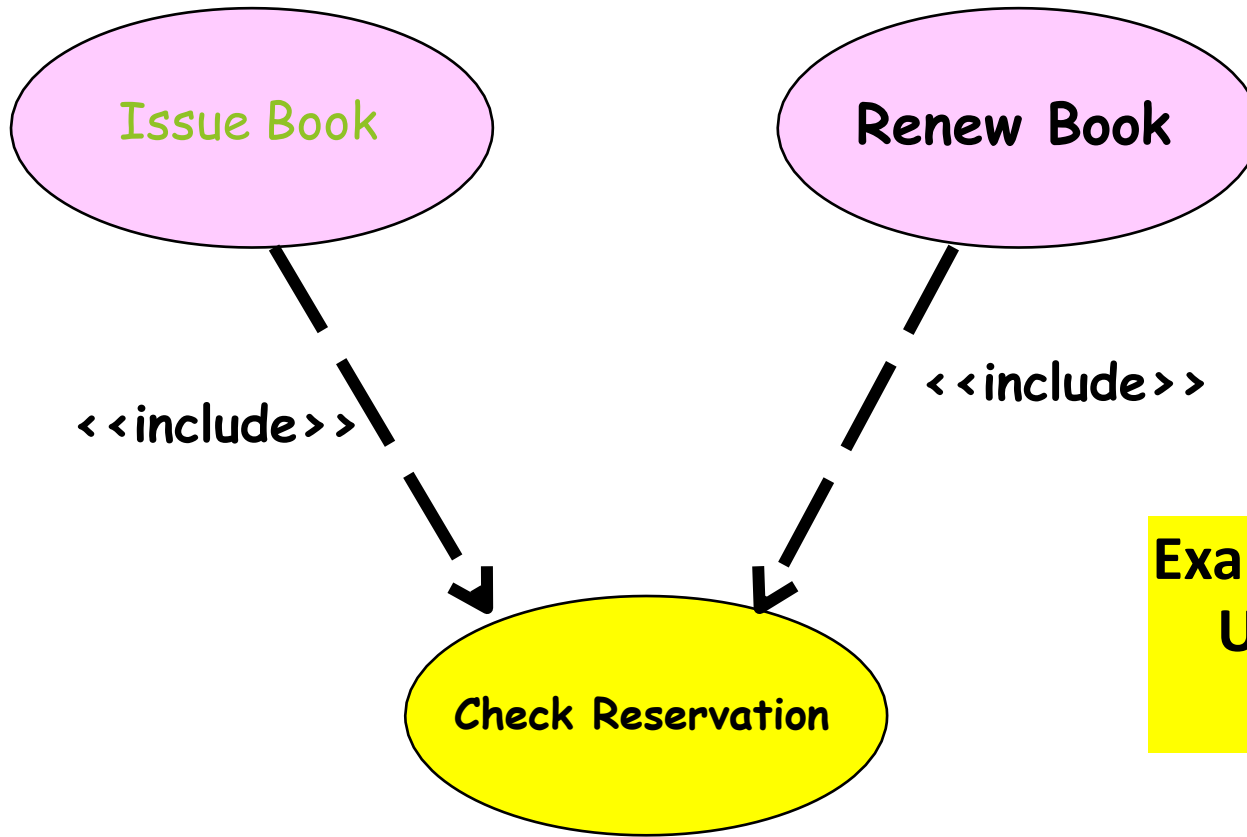
Factoring Use Cases Using Generalization





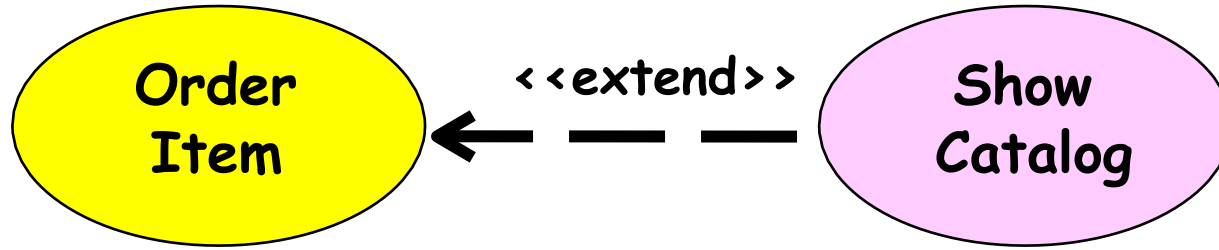
Factoring Use Cases Using Include

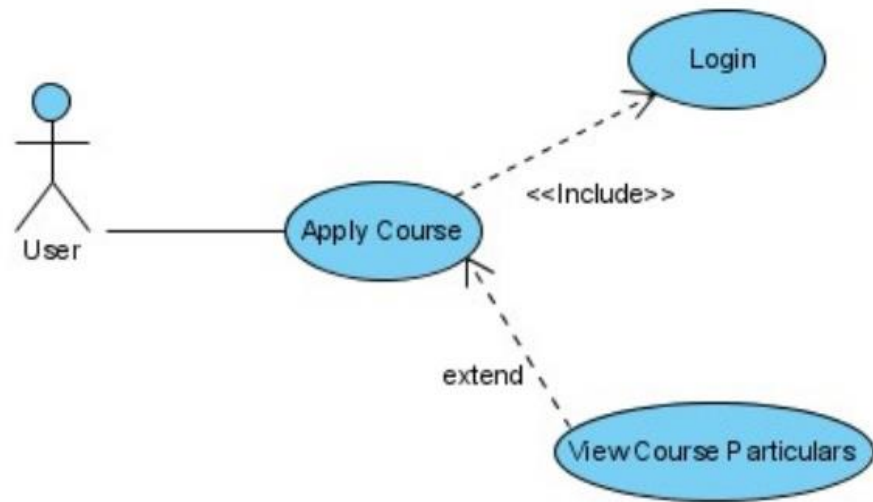




**Example of Factoring
Use Cases Using
Include**

Example Factoring A Use Case Using Extend





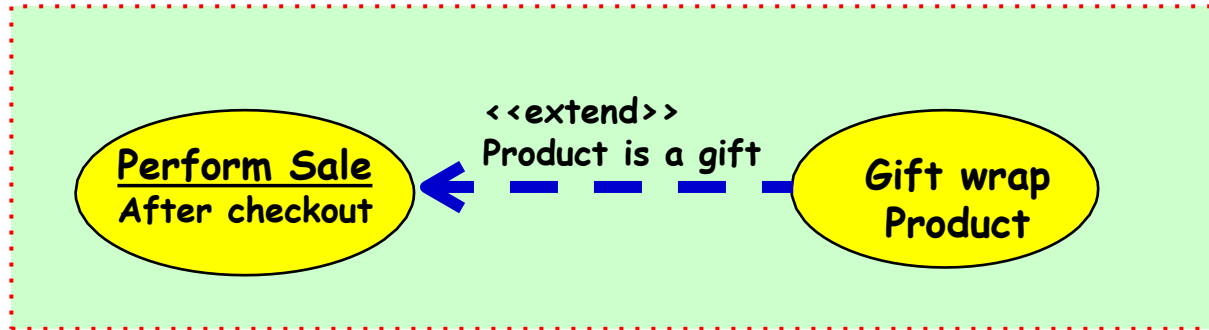
- Use case name should begin with a verb.
- While use cases do not explicitly imply timing:
 - Order use cases from top to bottom to imply timing -- it improves readability.
- **The primary actors should appear in the left.**
- Actors are associated with one or more use cases.
- Do not use arrows on the actor-use case relationship.
- **To initiate scheduled events include an actor called “time”, or “calendar”**
- **Do not show actors interacting with each other.**

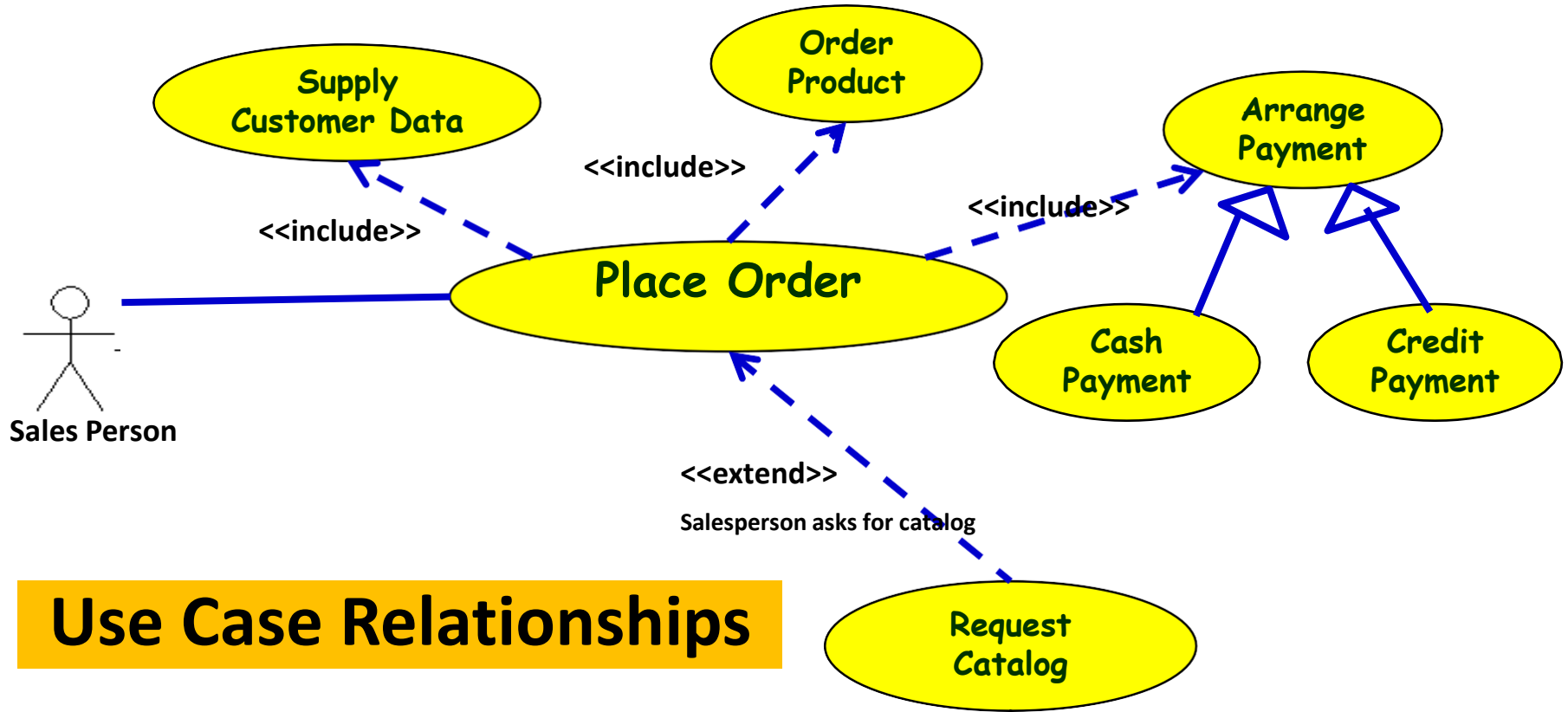
- Use cases should be named and organized from the perspective of the users.
- Use cases should start off simple and at as much high view as possible.
 - Can be refined and detailed further.
- Use case diagrams represent functionality:
 - **Should focus on the "what" and not the "how".**

Effective Use Case Modelling

Extension Point

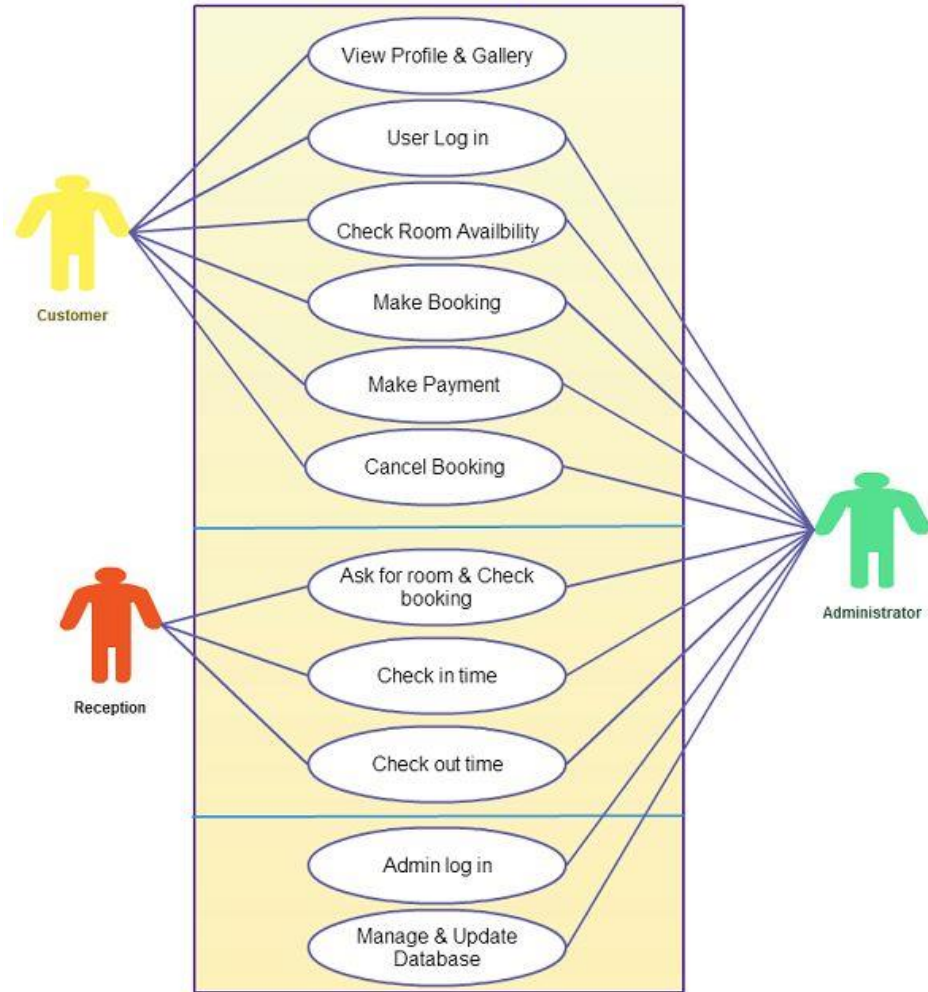
- The base use case may include/extend other use cases:
 - **At certain points, called extension points.**
- Note the direction of the arrow

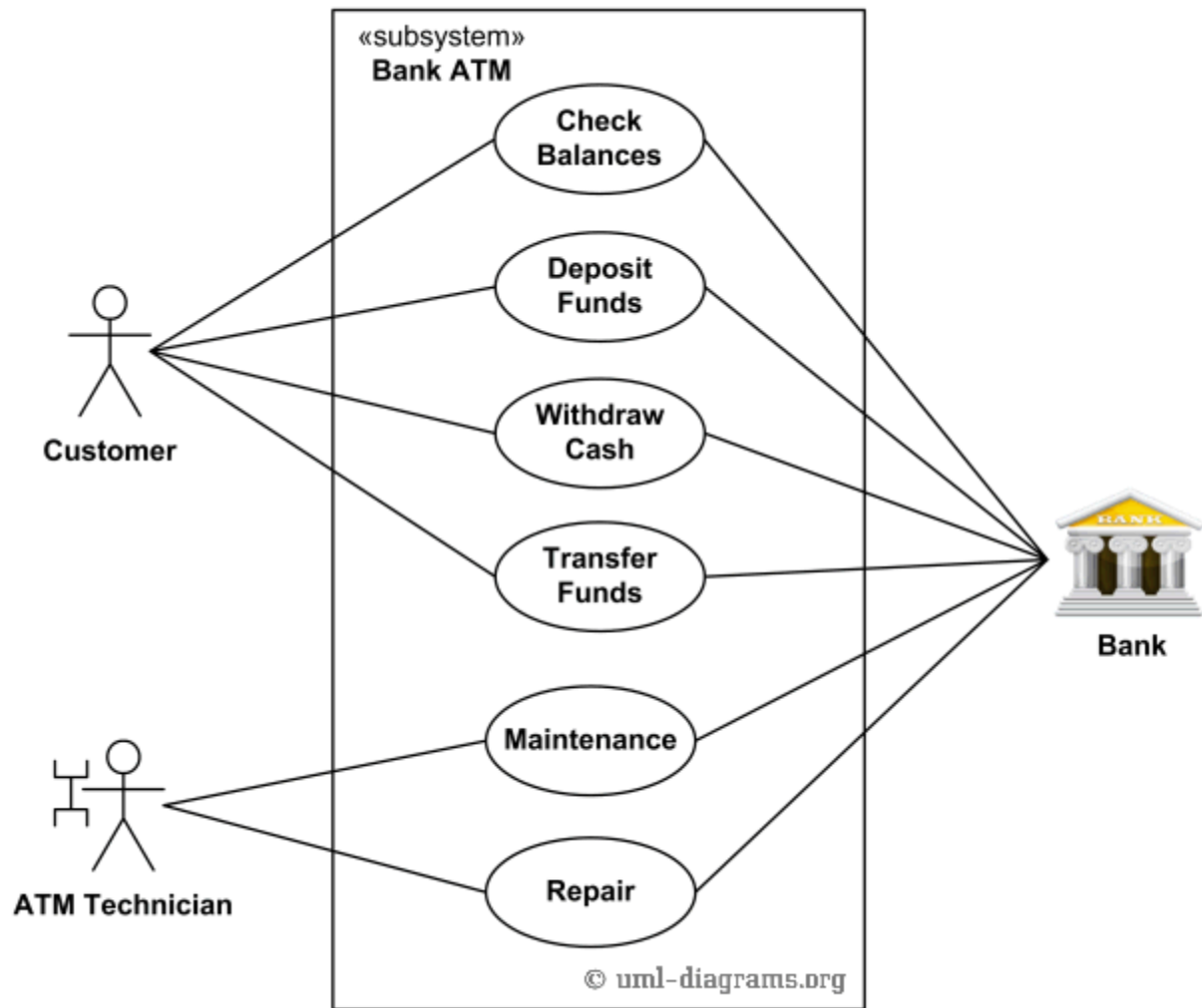


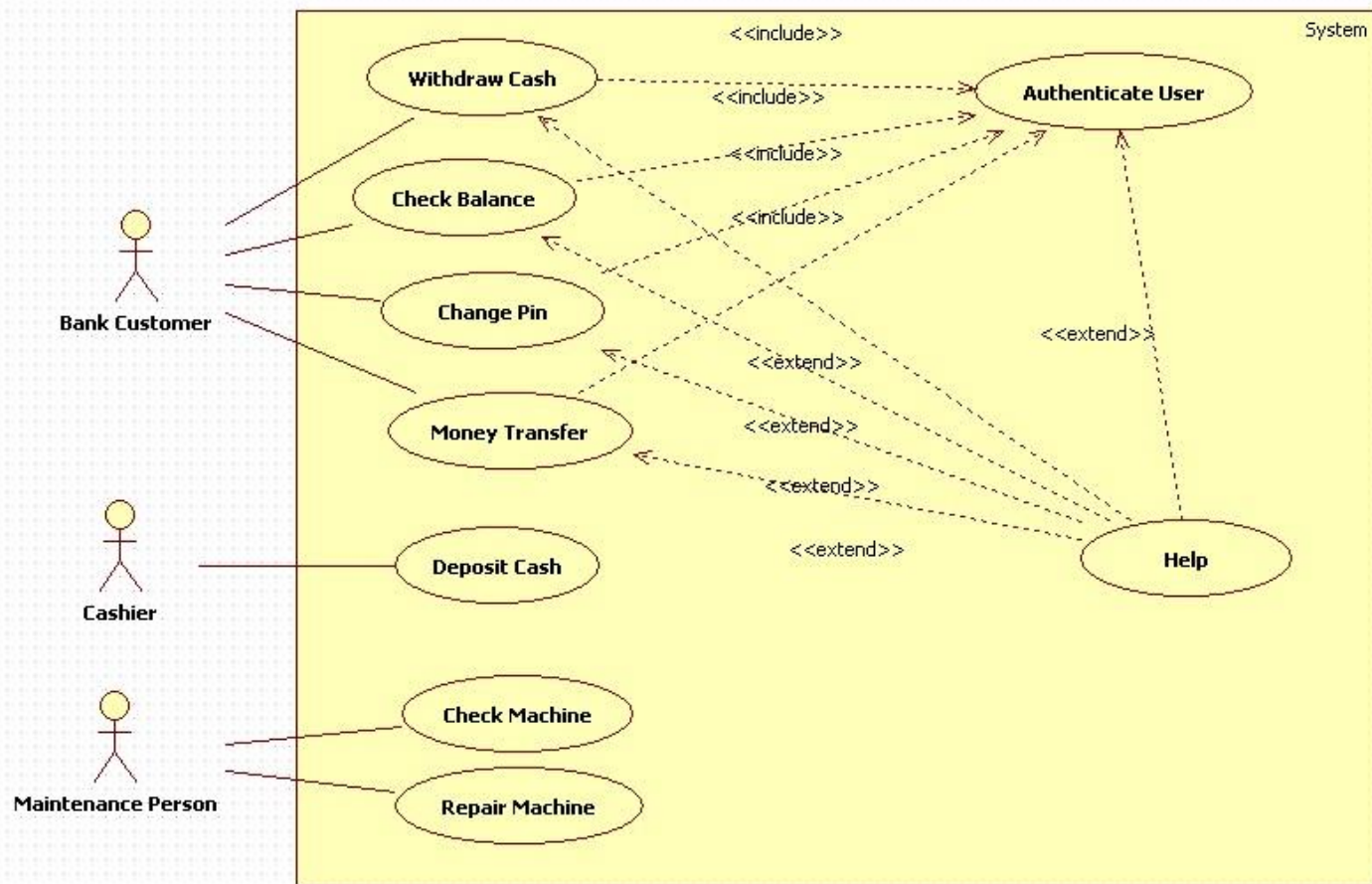


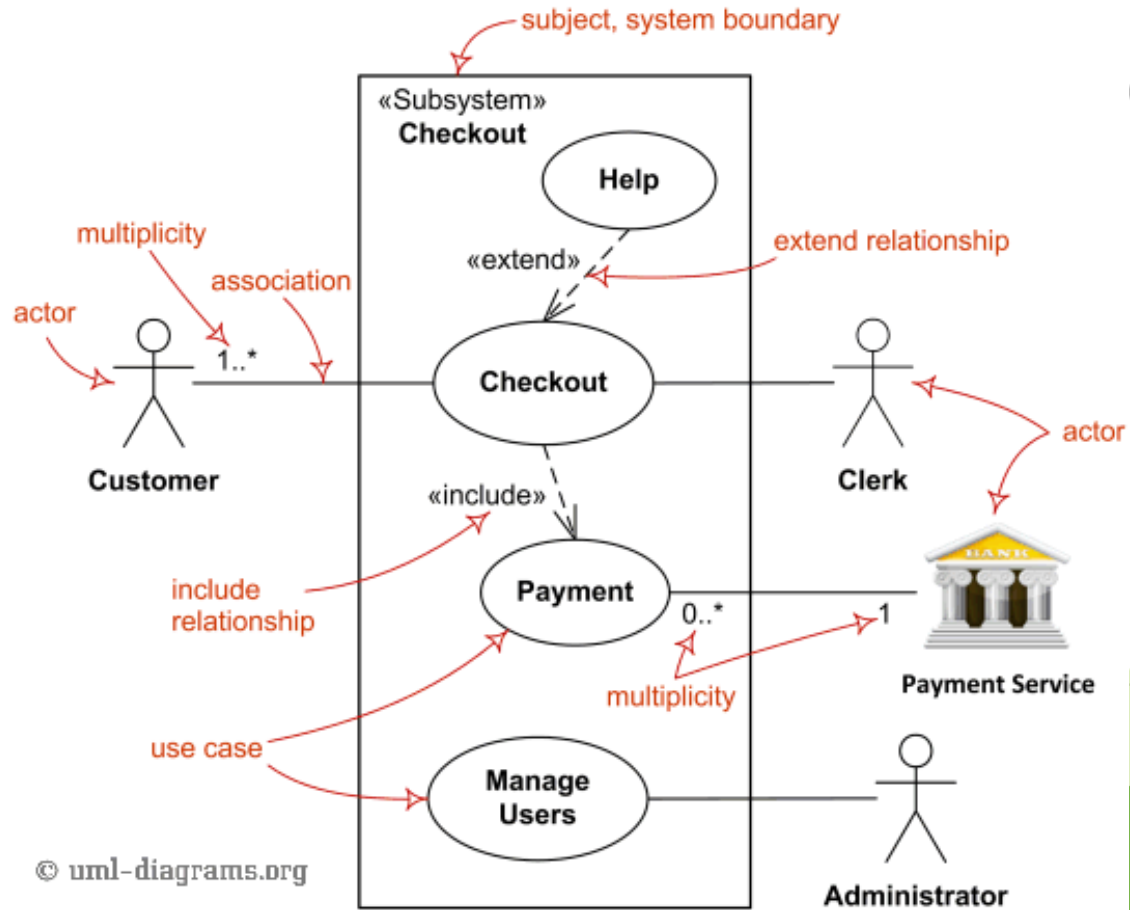
Use Case Relationships

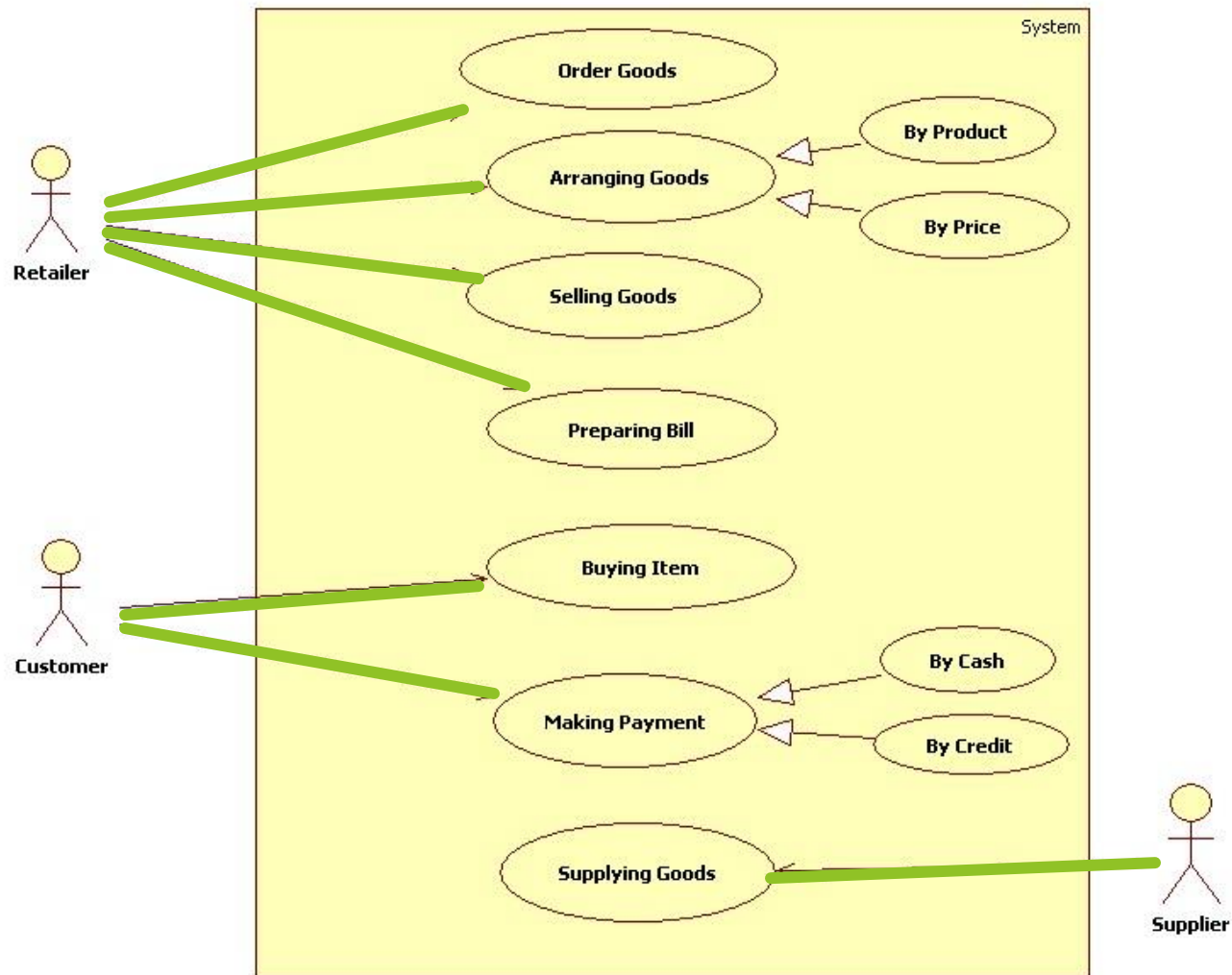
Hotel Management System

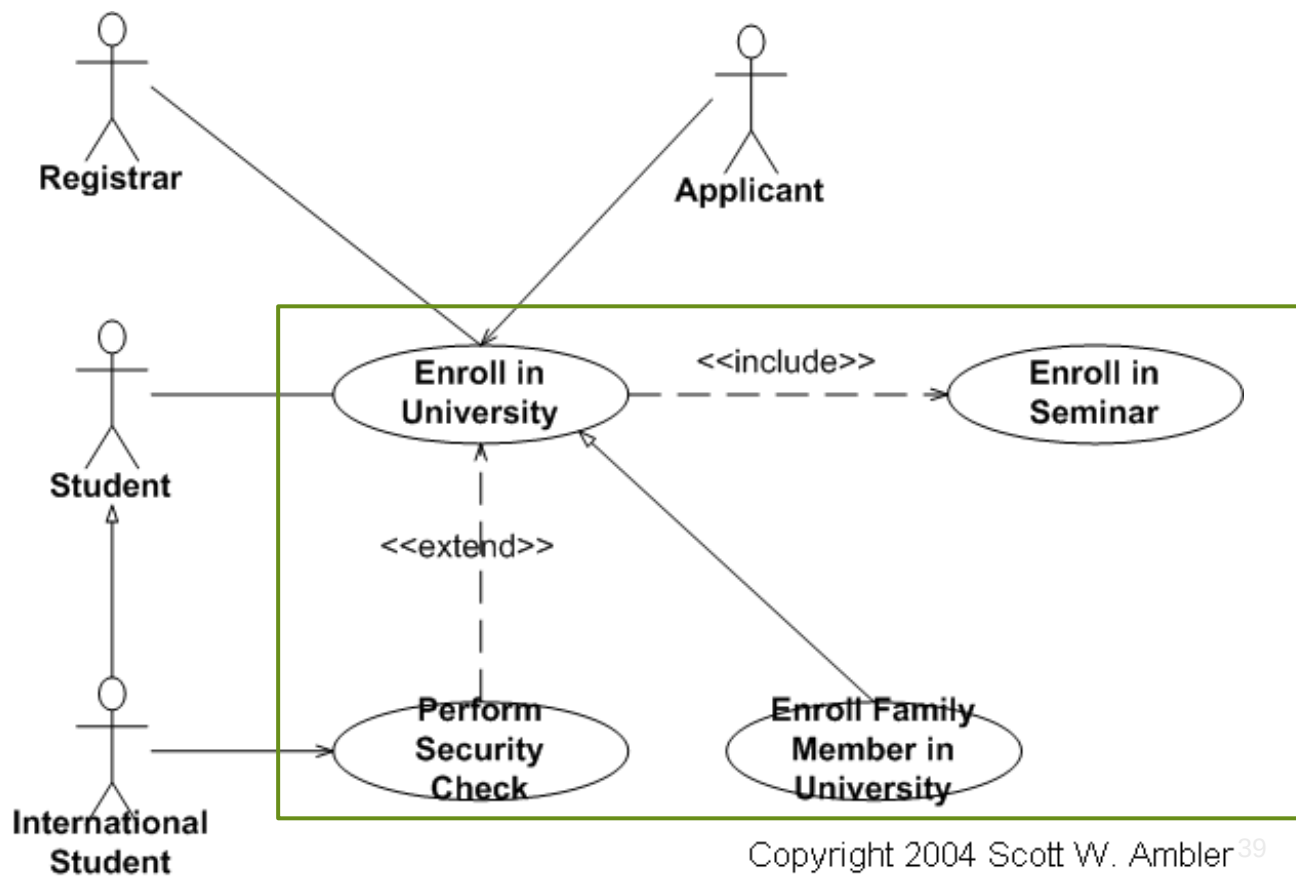


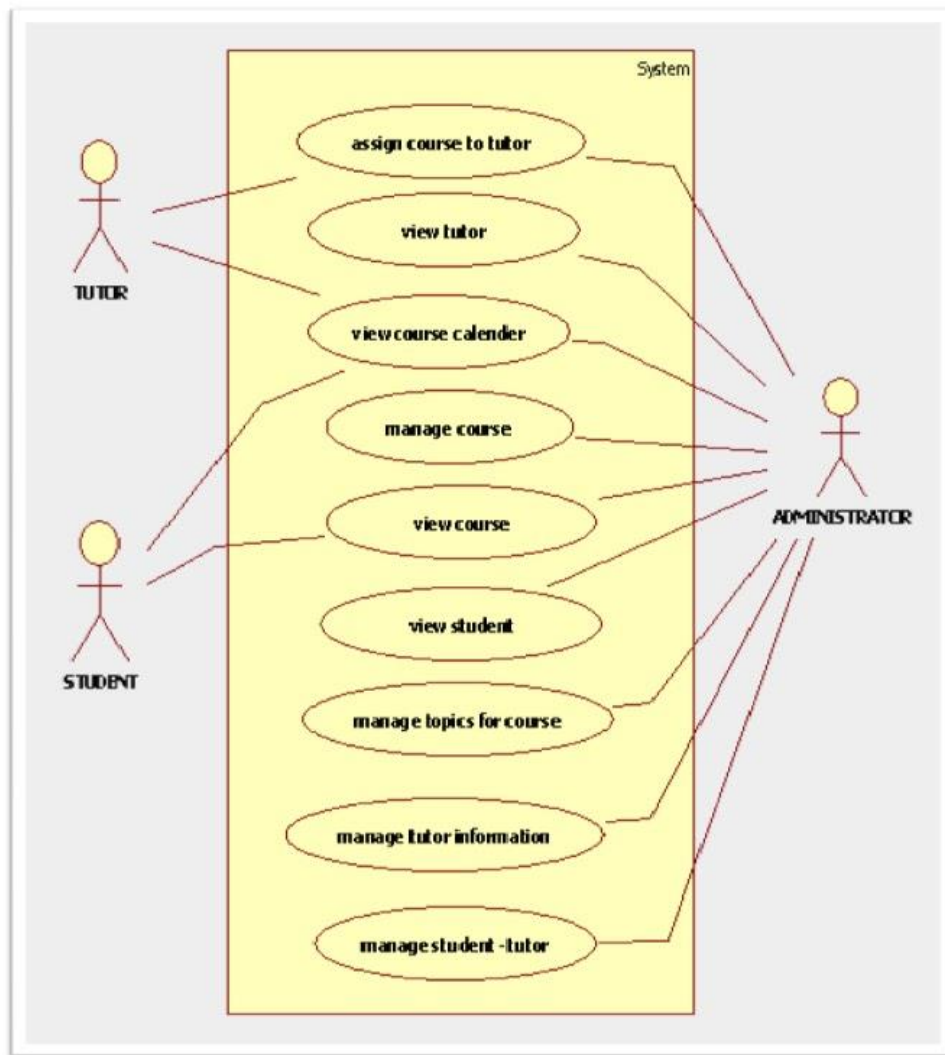












2.4 use case Diagram

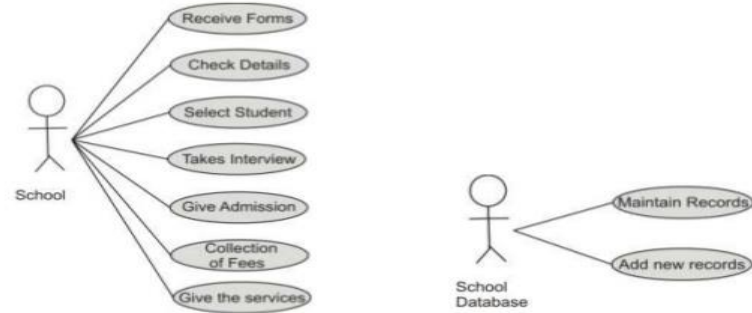


Fig-school use case diagram

Fig-school database use case diagram

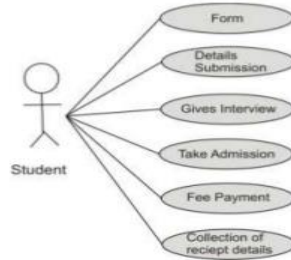


Fig-student use case diagram

Identification of Use Cases

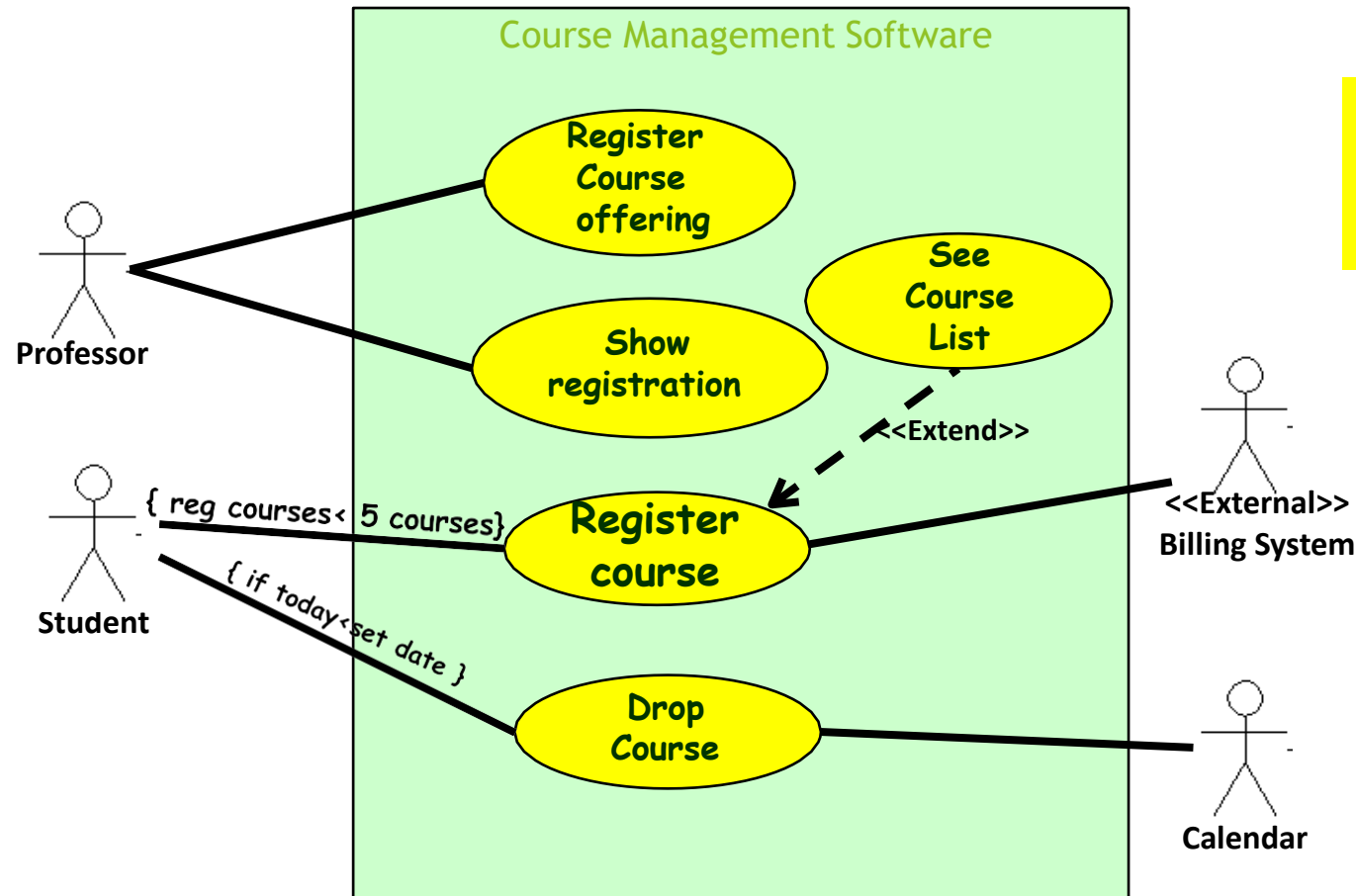
1. Actor-based:

- Identify the actors related to a system or organization.
- For each actor, identify the processes they initiate or participate in.

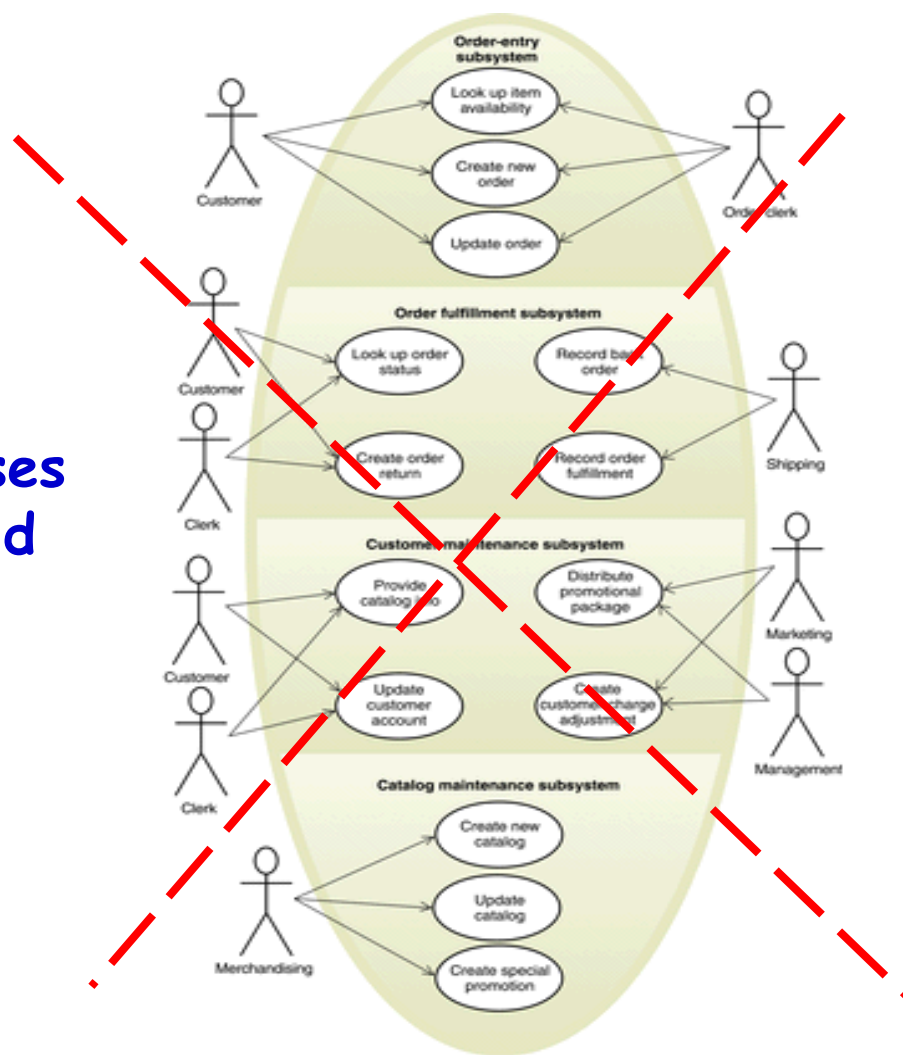
2. Event-based

- Identify the external events that the system must respond to.
- Relate the events to actors and use cases.

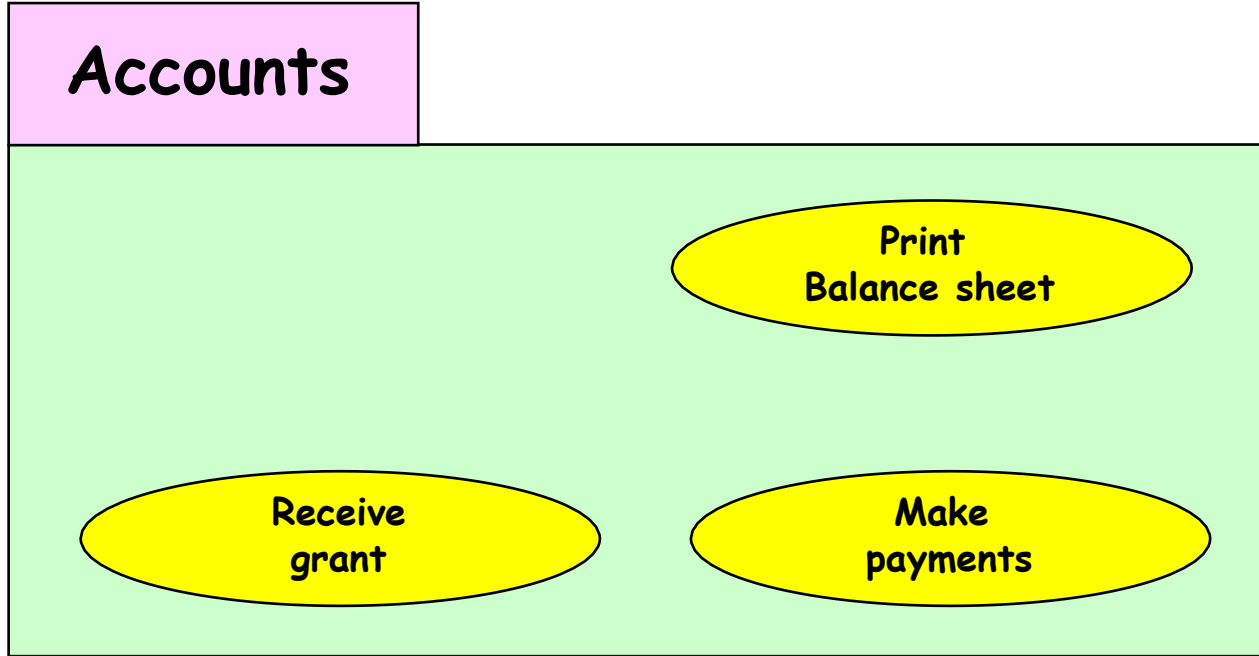
Example 2: Model Solution



Too many use cases
at any level should
be avoided!



Use Case Packaging

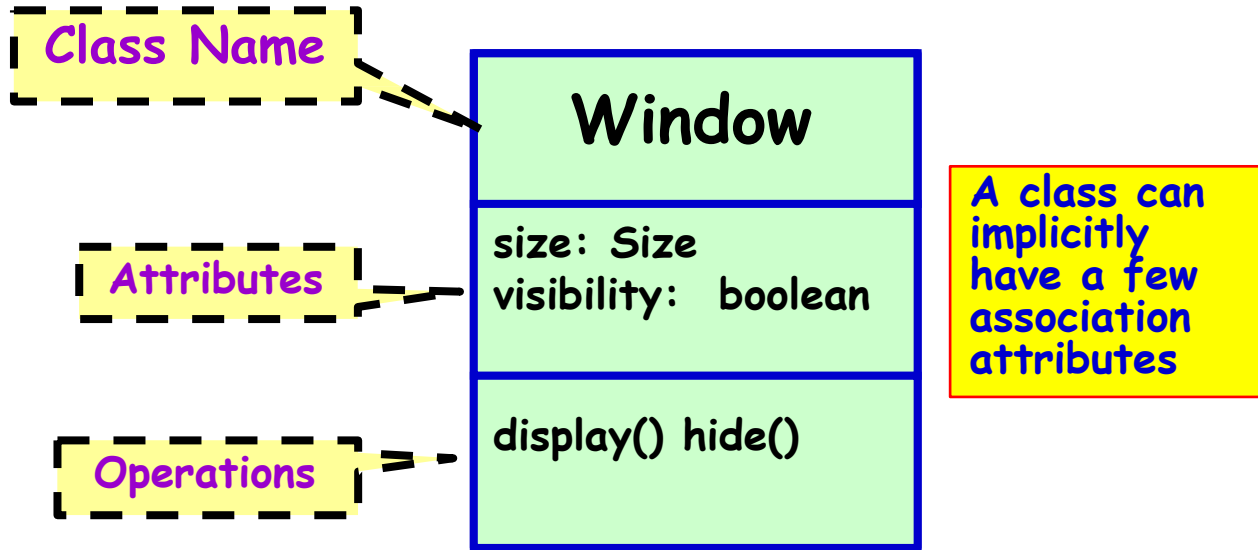


Class Diagram

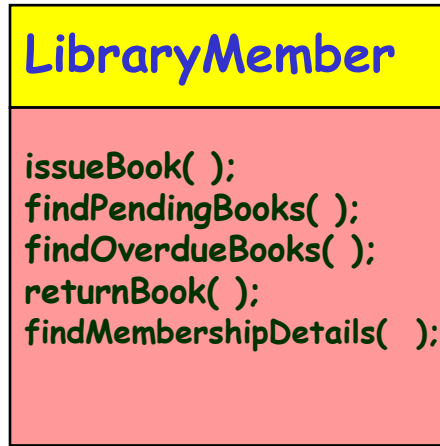
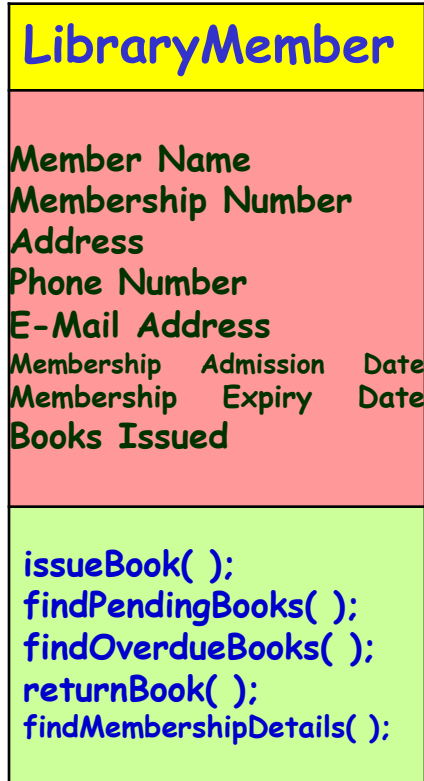
- Classes:
 - Entities with common features, i.e. attributes and operations.
 - Represented as solid outline rectangle with compartments.
 - Compartments for **name, attributes, and operations**.
 - Attribute and operation compartments are optional depending on the purpose of a diagram.

UML Class Representation

- A class represents a set of objects having similar attributes, operations, relationships and behavior.



Different representations of the LibraryMember class



**Example UML
Classes**

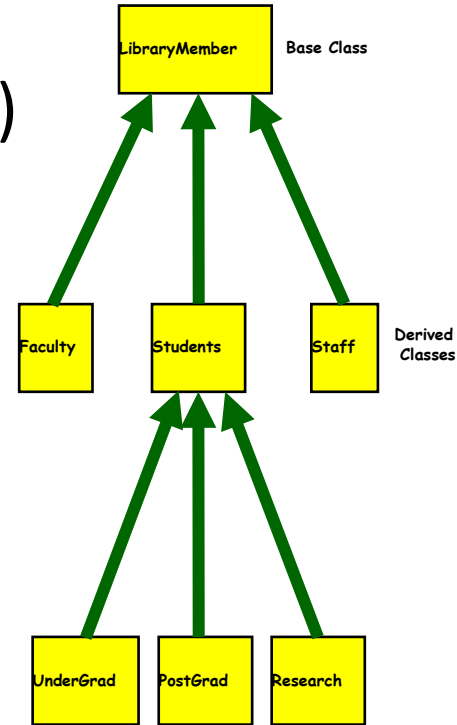
What are the Different Types of Relationships

Among Classes?

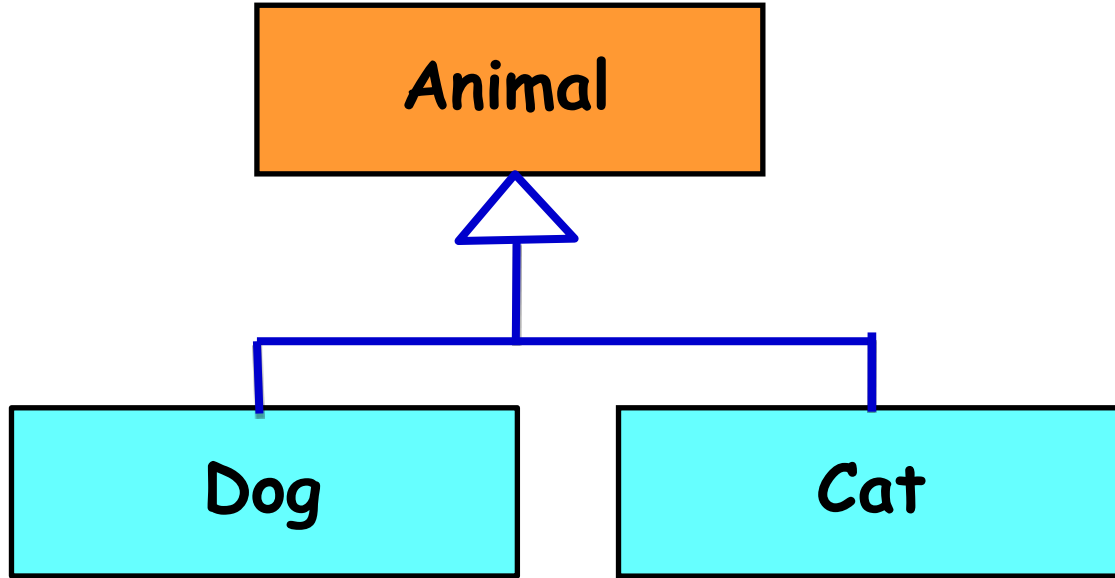
- Four types of relationships:
 - **Inheritance**
 - **Association**
 - **Aggregation/Composition**
 - **Dependency**

Inheritance

- Allows to define a new class (derived class) by extending an existing class (base class).
 - Represents generalization-specialization
 - Allows redefinition of the existing methods (method overriding).

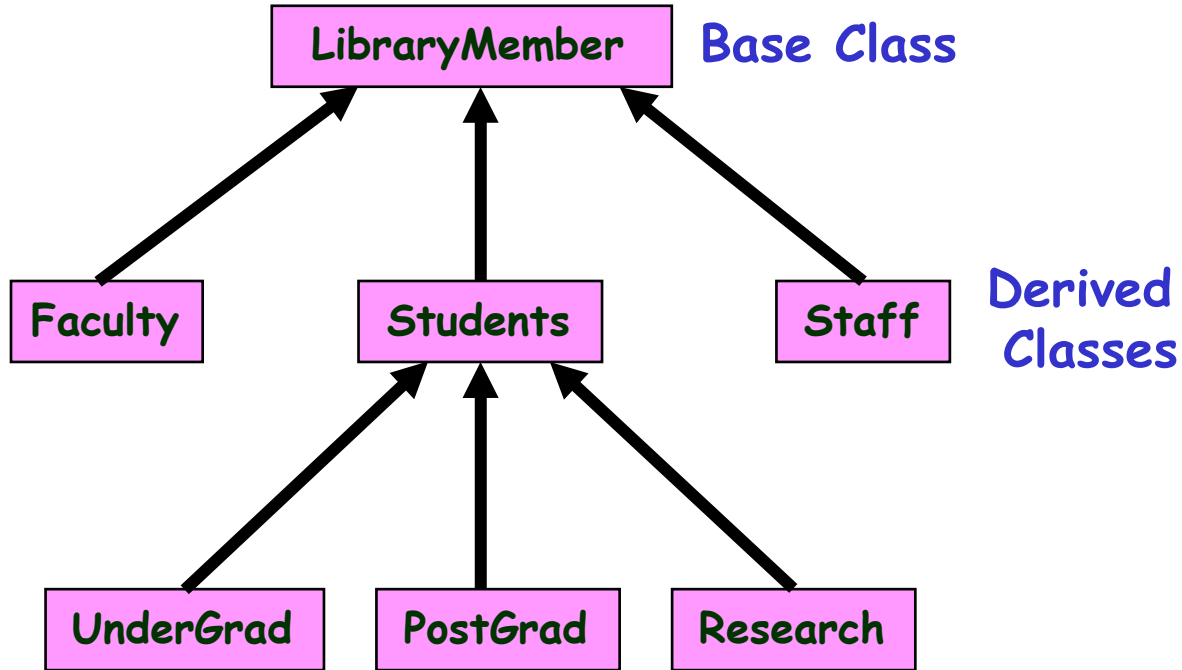


Inheritance Example

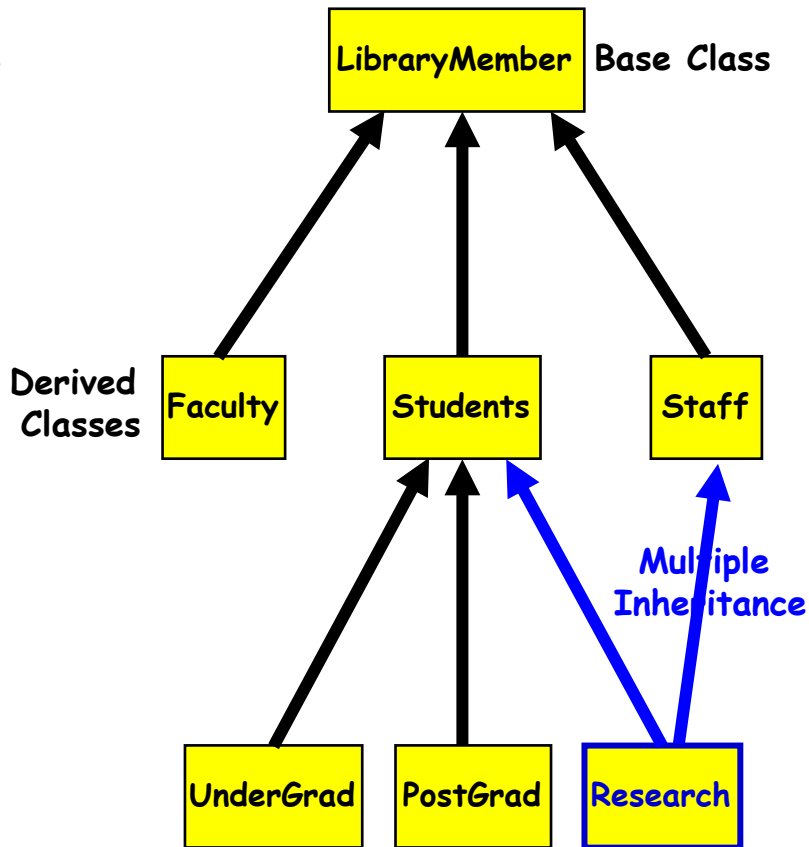
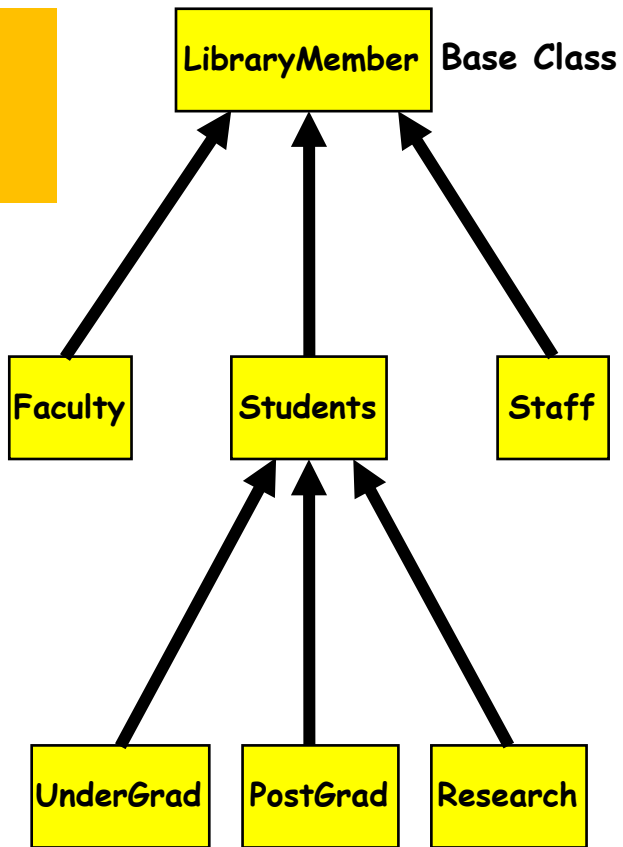


Inheritance

- Lets a subclass inherit attributes and methods from a base class.



Multiple Inheritance



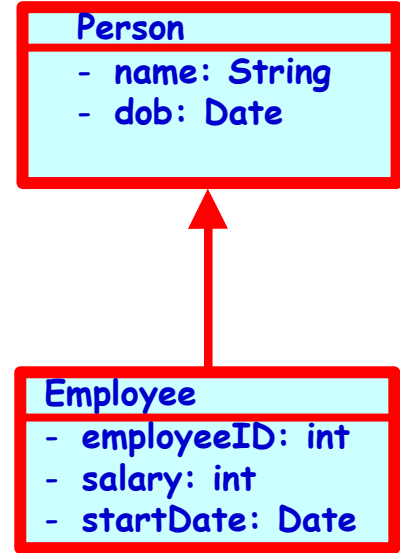
Inheritance Implementation in Java

- Inheritance is declared using the "extends" keyword
 - Even when no inheritance defined, the class implicitly extends a class called Object.

```
class Person{  
    private String name;  
    private Date dob;  
    ...  
}
```

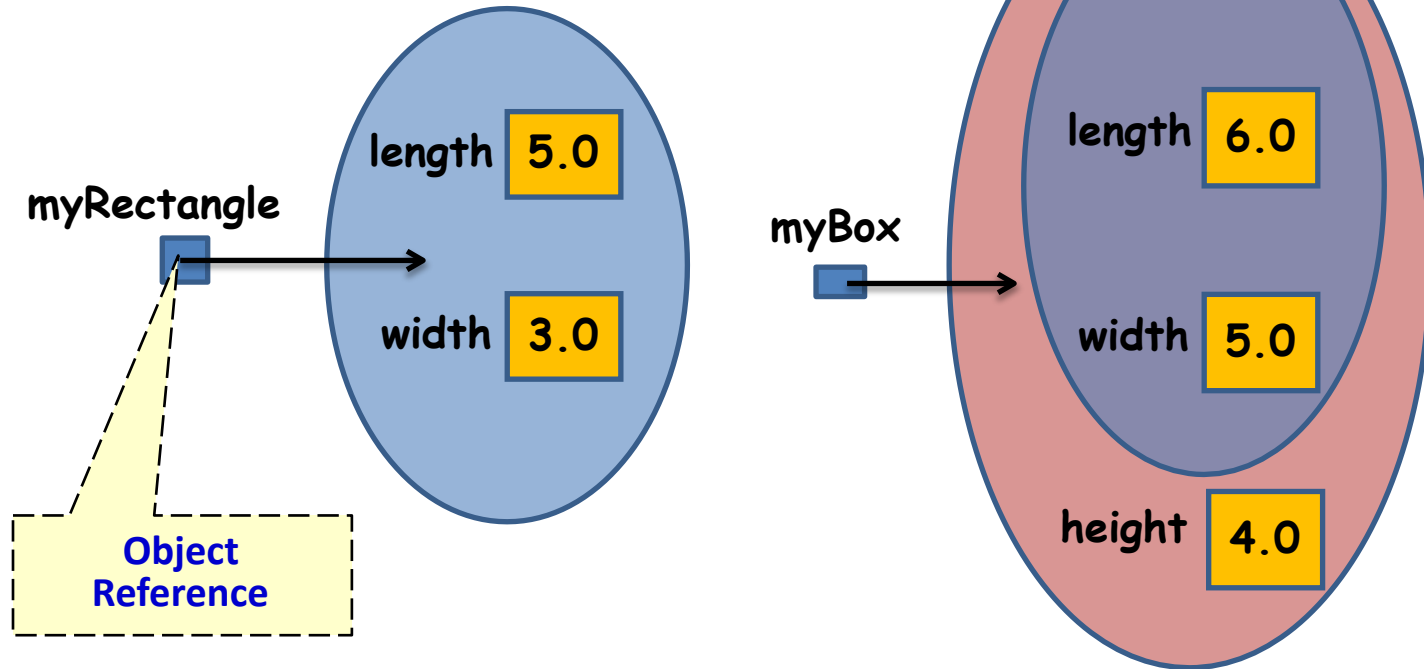
```
class Employee extends Person{  
    private int employeeID;  
    private int salary;  
    private Date startDate;  
    ...  
}
```

```
Employee anEmployee = new Employee();
```

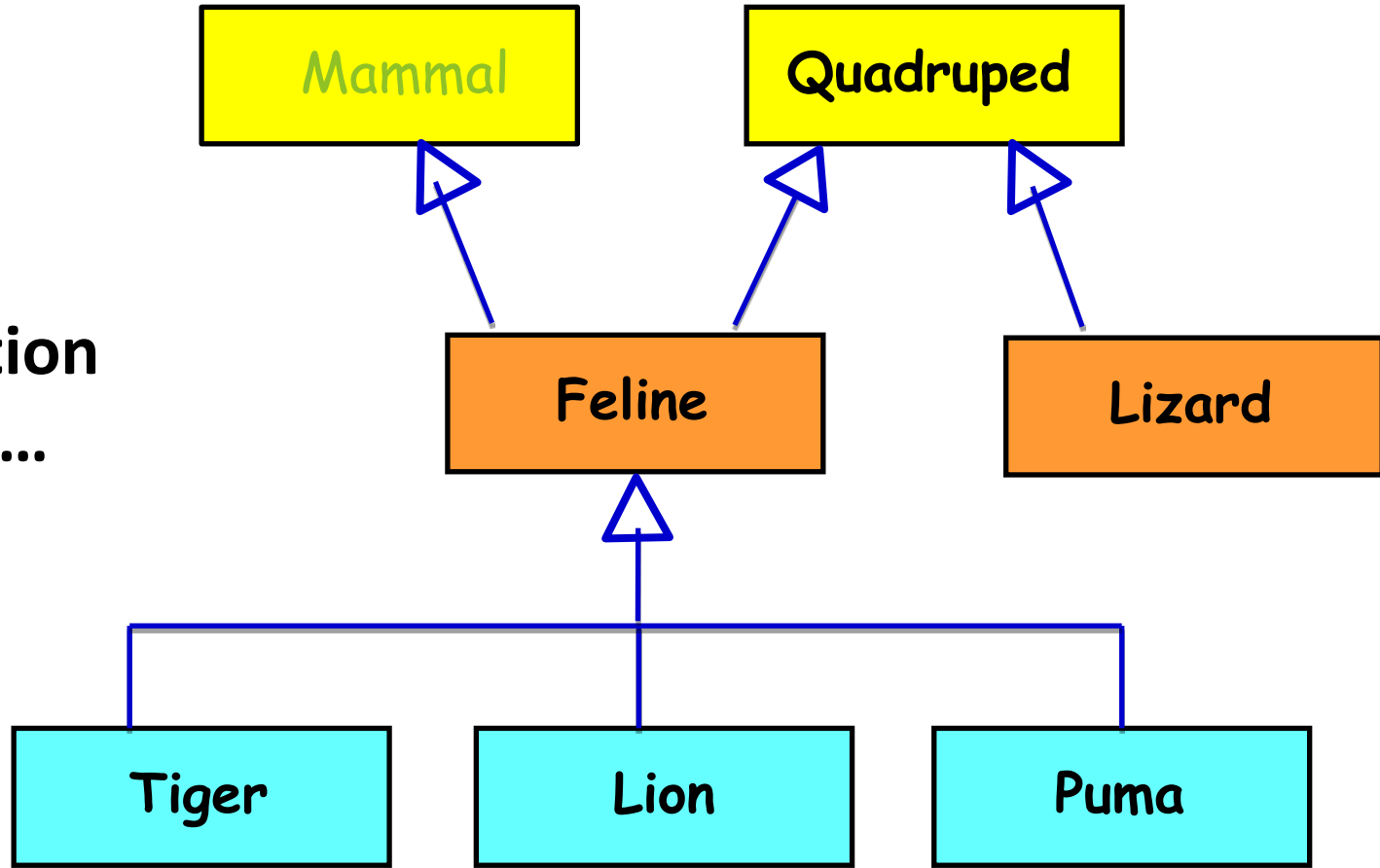


Objects myRectangle and myBox

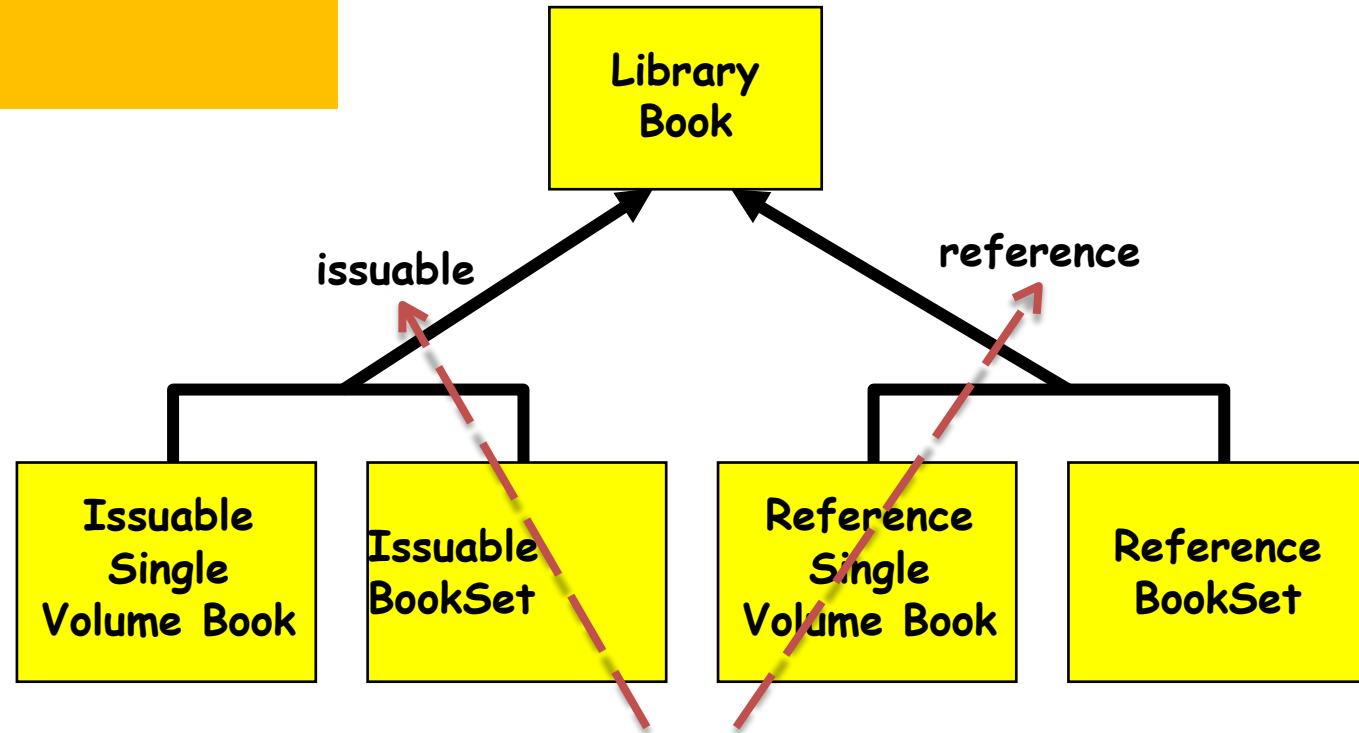
```
Rectangle myRectangle = new Rectangle(5, 3);  
Box myBox = new Box(6, 5, 4);
```



**More
Generalization
Examples...**



Inheritance Example



Discriminator: allows one to group subclasses into clusters that correspond to a semantic category.

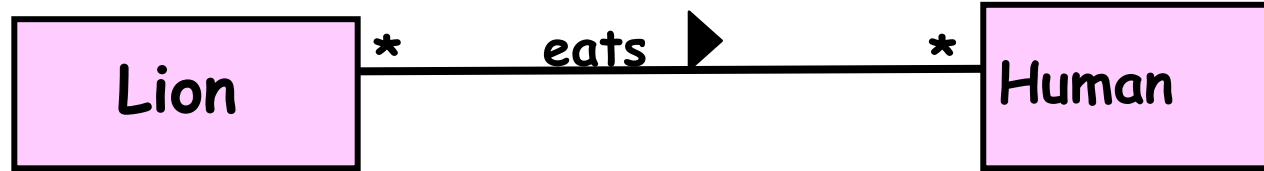
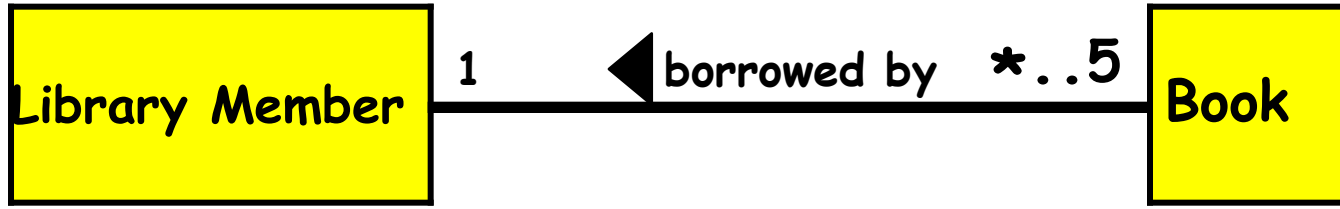
Inheritance Pitfalls

- Inheritance certainly promotes reuse.
- **Indiscriminate use can result in poor quality programs.**
- Base class attributes and methods visible in derived class...
 - Leads to tight coupling

Association Relationship

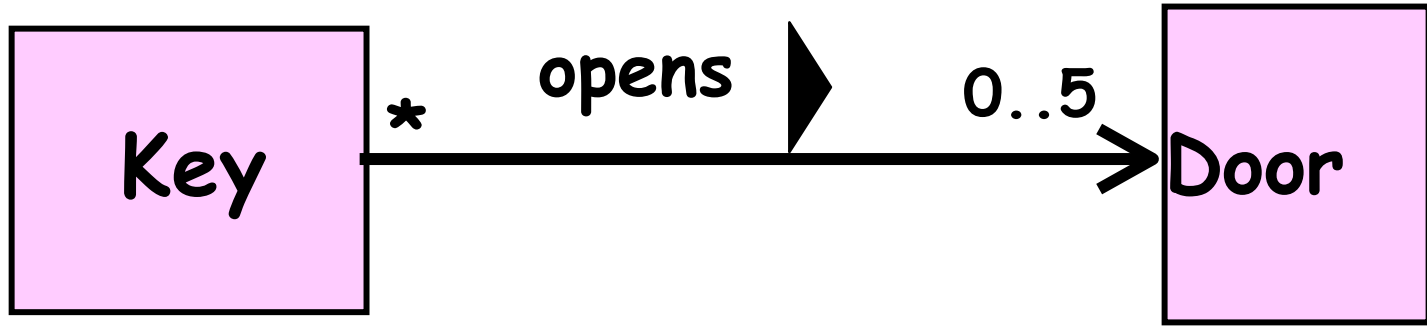
- How implemented in program?
- Enables objects to communicate with each other:
 - One object must “know” the ID of the corresponding object in the association.
- Usually binary:
 - But in general can be n-ary.

Association - More Examples



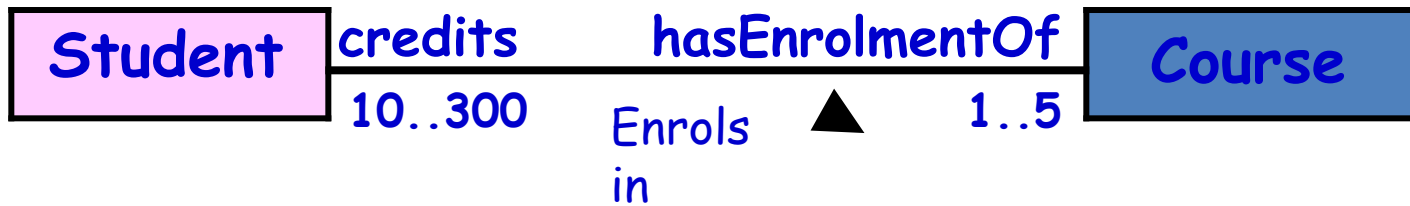
Multiplicity: The number of objects from one class that relate with a single object in an associated class.

Navigability

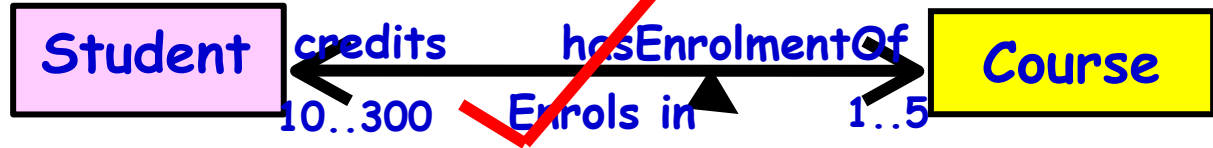
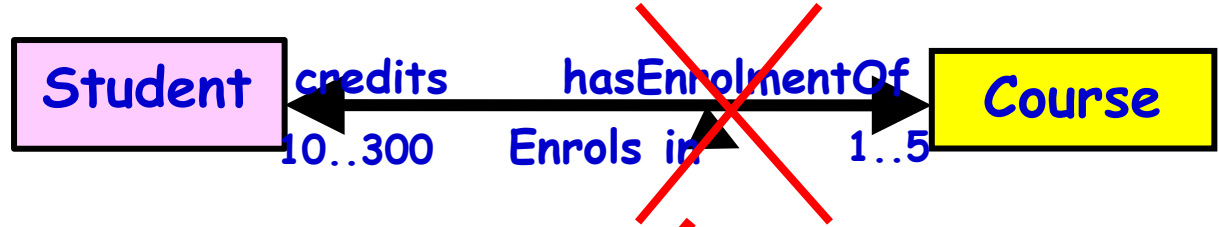
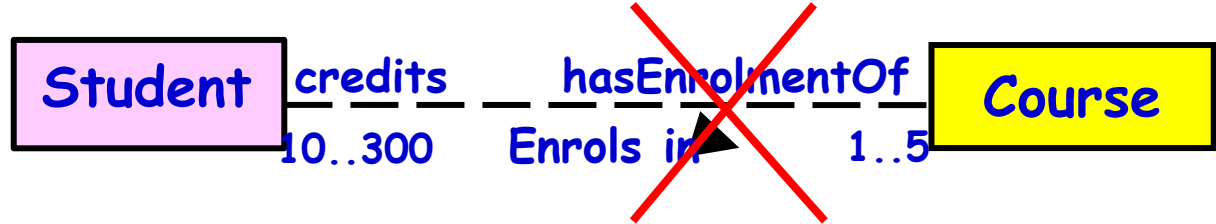
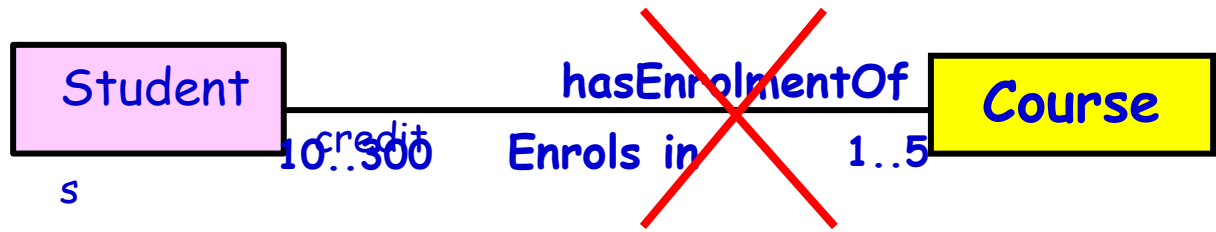


Quiz: Draw Class Diagram

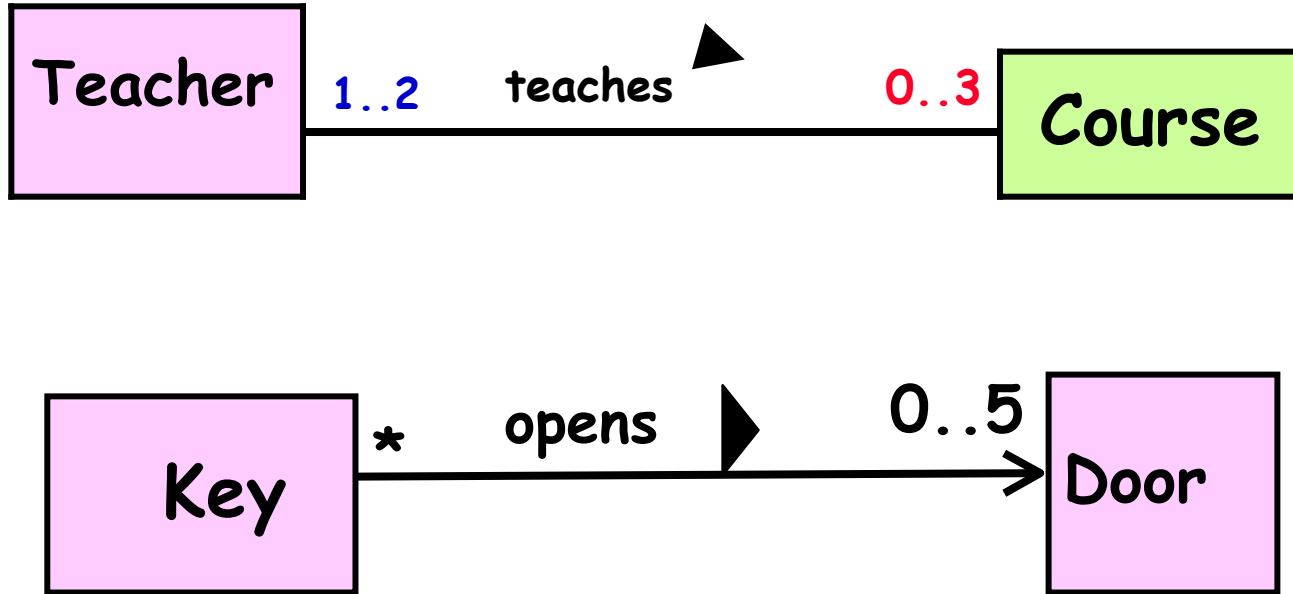
- A Student can take up to five Courses.
- A student needs to enroll in at least one course.
- Up to 300 students can enroll in a course.
- An offered subject in a semester should have at least 10 registered students.



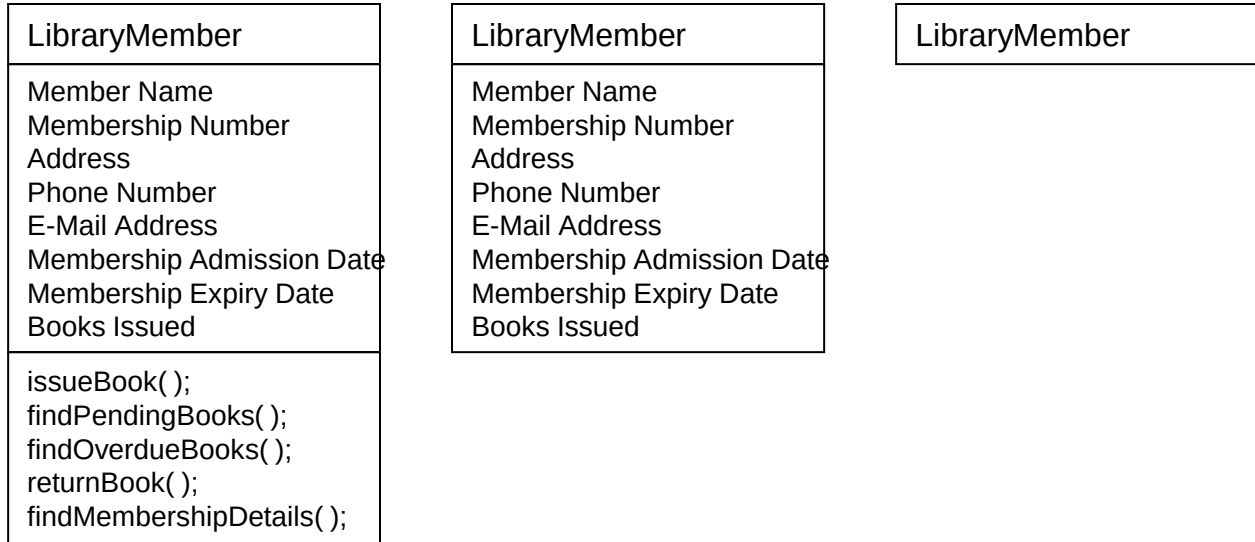
Identify as
Correct or
Wrong



Quiz: Read the Diagram

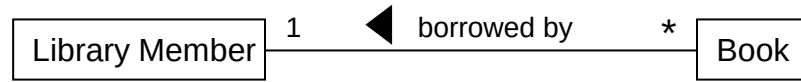


Example of Class diagram



Different representations of the `LibraryMember` class

Association Relationship

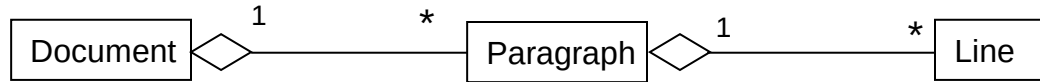


Association between two classes

Aggregation Relationship

- ▶ Represent a whole-part relationship
- ▶ Represented by **diamond** symbol at the composite end
- ▶ Cannot be reflexive(i.e. recursive)
- ▶ Not symmetric
- ▶ It can be transitive

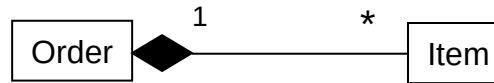
Aggregation Relationship



Representation of aggregation

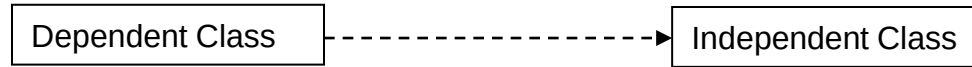
Composition Relationship

- Life of item is same as the order



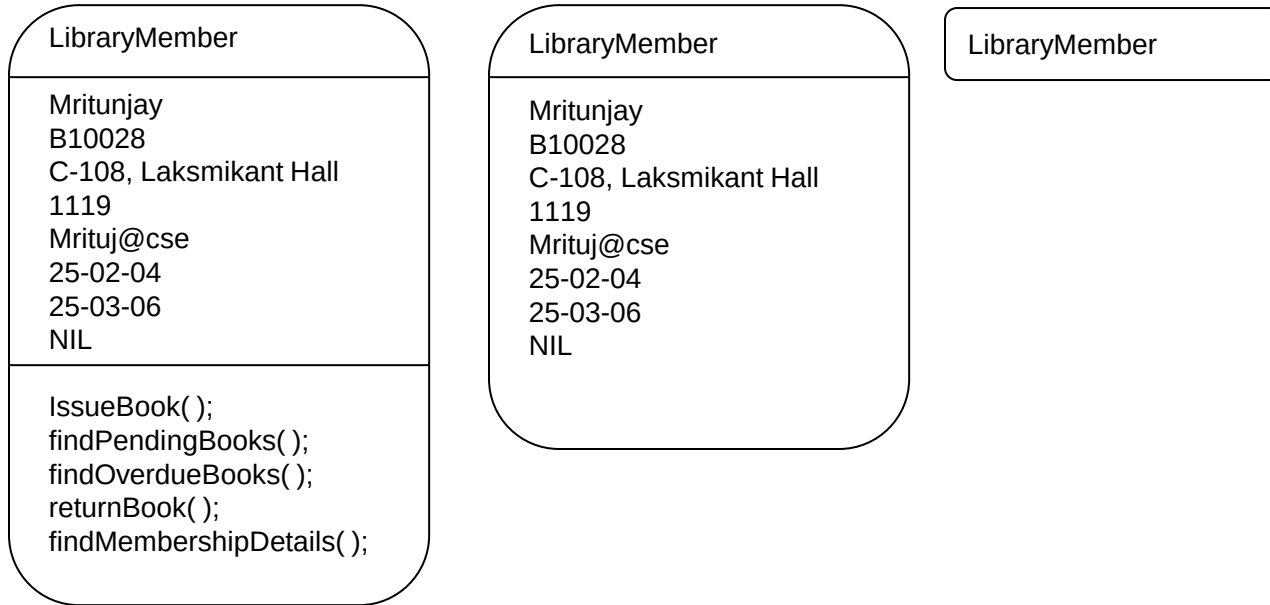
Representation of composition

Class Dependency



Representation of dependence between class

Object diagram



Different representations of the `LibraryMember` object

Interaction diagram

- ▶ Models how groups of objects collaborate to realize some behaviour
- ▶ Typically each interaction diagram realizes behaviour of a single use case

Interaction diagram

- ▶ Two kinds: **Sequence** & **Collaboration**
- ▶ Two diagrams are equivalent but portrays different perspective
- ▶ These diagram play a very important role in the design process

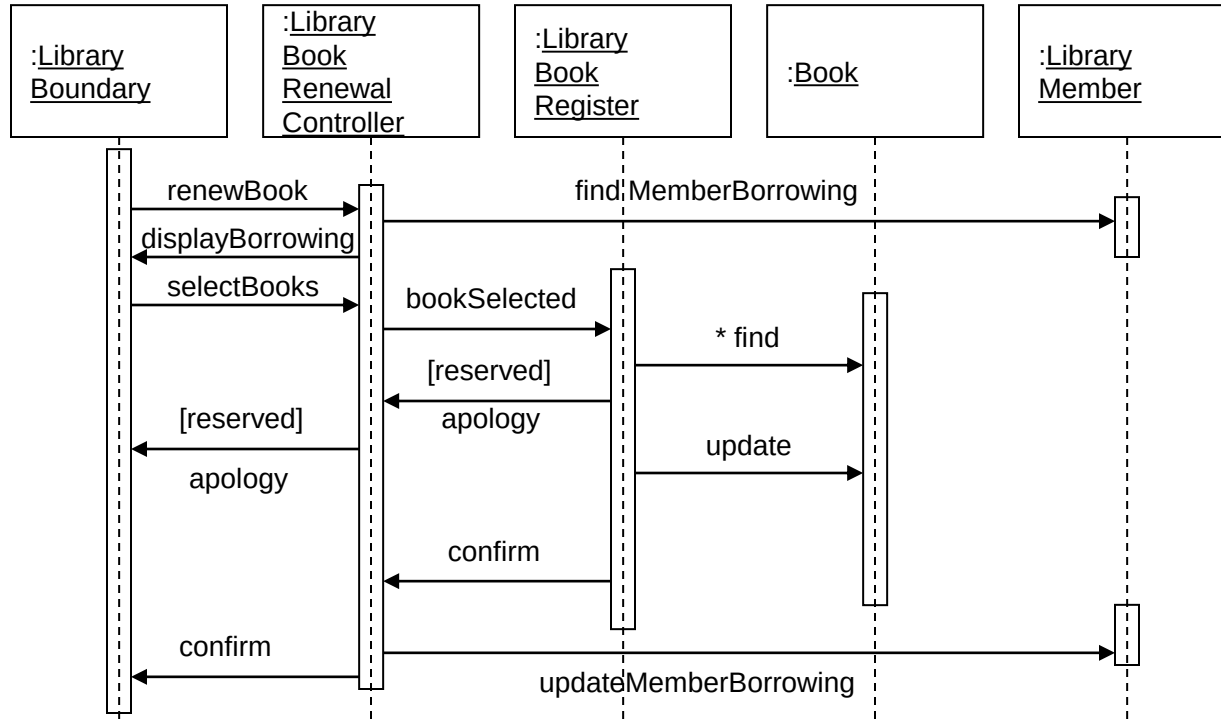
Sequence diagram

- ▶ Shows interaction among objects as two-dimensional chart
- ▶ **Objects** are shown as **boxes** at top
- ▶ If object created during execution then shown at appropriate place
- ▶ **Objects existence** are shown as **dashed lines** (lifeline)
- ▶ **Objects activeness**, shown as **rectangle** on lifeline

Sequence diagram

- ▶ **Messages** are shown as **arrows**
- ▶ Message labelled with message name
- ▶ Message can be labelled with **control information**
- ▶ Two types of control information: **condition** ([]) & an **iteration** (*)

Example of Sequence diagram

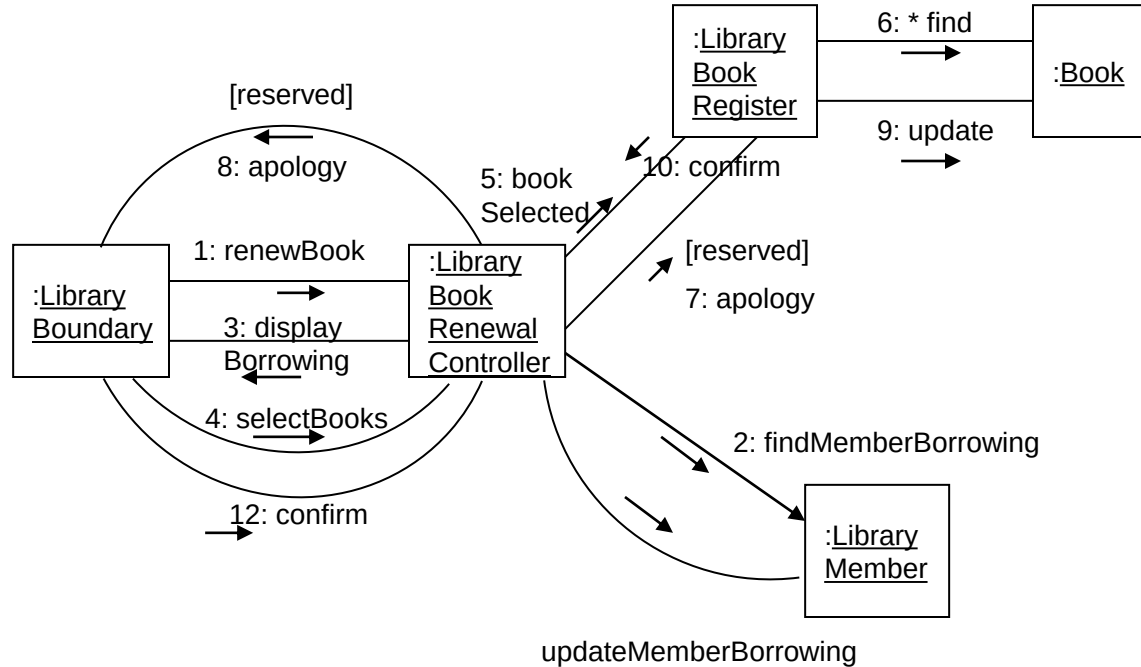


Sequence Diagram for the renew book use case

Collaboration diagram

- ▶ Shows both **structural** and **behavioural** aspects
- ▶ Objects are **collaborator**, shown as boxes
- ▶ Messages between objects shown as a **solid line**
- ▶ Message is shown as a **labelled arrow** placed near the link
- ▶ Messages are prefixed with **sequence numbers** to show relative sequencing

Example of Collaboration diagram



Collaboration Diagram for the renew book use case

Activity diagram

- ▶ New concept, possibly based on event diagram of **Odell** [1992]
- ▶ Represent processing activity, may not correspond to methods
- ▶ Activity is a state with an internal action and one/many outgoing transition

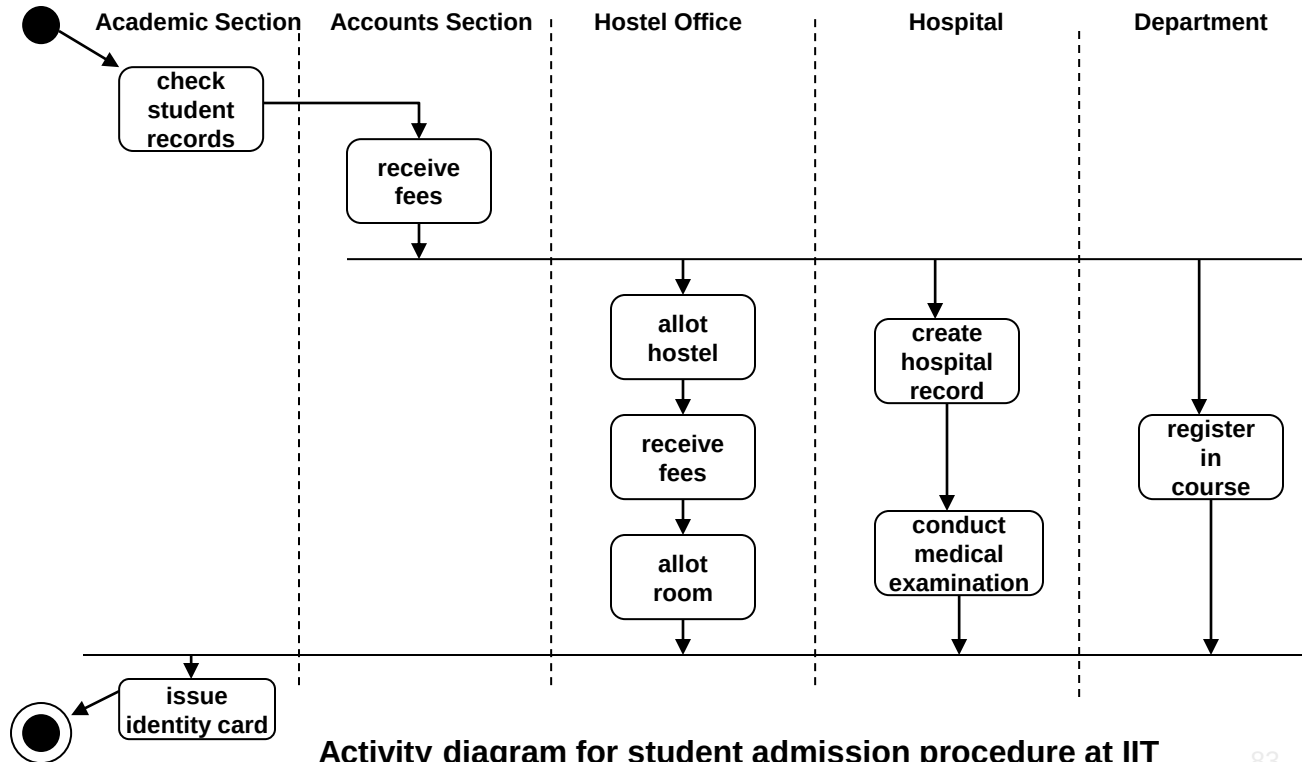
Activity diagram

- ▶ Can represent parallel activity and synchronization aspects
- ▶ **Swim lanes** enable to group activities based on who is performing them
- ▶ Example: academic department vs. hostel

Activity diagram

- ▶ Normally employed in business process modelling
- ▶ Carried out during requirement analysis and specification
- ▶ Can be used to develop interaction diagrams

Example of Activity diagram



Activity diagram for student admission procedure at IIT

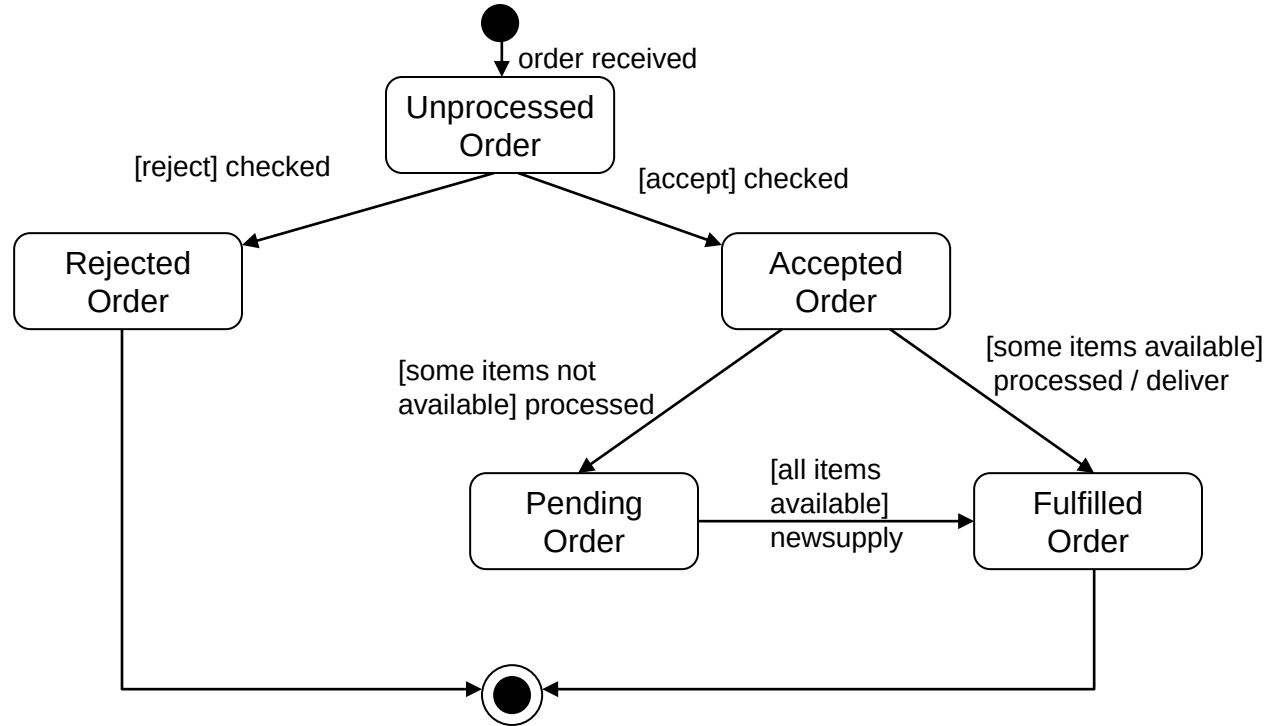
State Chart diagram

- ▶ Based on the work of **David Harel** [1990]
- ▶ Model how the state of an object changes in its lifetime
- ▶ Based on finite state machine (FSM) formalism

State Chart diagram

- ▶ Elements of state chart diagram
- ▶ **Initial State:** Filled circle
- ▶ **Final State:** Filled circle inside larger circle
- ▶ **State:** Rectangle with rounded corners
- ▶ **Transitions:** Arrow between states, also boolean logic condition (**guard**)

Example of State Chart diagram



Example: State chart diagram for an order object